목차

React와 함께하는 인프라 서비스 개발 및 활용

목차

1. React 소개
2. 기초 문법
3. props
4. 이벤트
5. useEffect
6. 컴포넌트 분리
7. API
8. Go 백엔드
9. React API 연동
10. LLM Chat
11. Podman/K8s

소개

과정

강사

소개

이 교육은 React 그리고, 컨테이너를 활용한 간단한 서비스 제작 교육이다. 실제로 React/NodeJS 그리고 JavaScript는 단기간에 학습 및 습득은 어렵다.

이러한 이유로 해당 교육에서는 다음과 같은 내용을 가지고 진행한다.

- NodeJS와 React 관계
- React 기반의 문법 및 간단한 애플리케이션 활용
- React 기반의 애플리케이션 패키징 및 배포
- 인프라 엔지니어 위한 도구 개발 과정(포드만/LLM)

소개

01

React 처음 사용자

02

컨테이너 기반으로 패키징 및 서비스배포

03

인프라에서 React 응용 및 활용이 궁금한 인프라 엔지니어

소개

과정

담당자 및 강사

강의일정

강의 일정은 아래와 같이 진행이 될 예정 입니다.

DAY 1: React/NodeJS 소개

- 왜 인프라 엔지니어가 React를 알아야 하나
- React 기본 구조
- JSX/state/props/component
- 미니 프로젝트 1: 사용자 카드 대시보드

강의일정

DAY 2: React 기반으로 서비스 구성 + 포드만

- event/useEffect
- metrics 대시보드
- API 필요성
- Go 백엔드 health/hello/metrics
- React에서 fetch 연동
- 미니 프로젝트 2: React API Dashboard

강의일정

DAY 3: 포드만으로 서비스 배포

- CPU 기반 LLM + React Chat
- llama.cpp 시나리오
- Podman 패키징
- K8s 배포 개요
- 최종 프로젝트 통합

강사

강사

강사

이름: 최국현

메일: tang@linux.com, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.

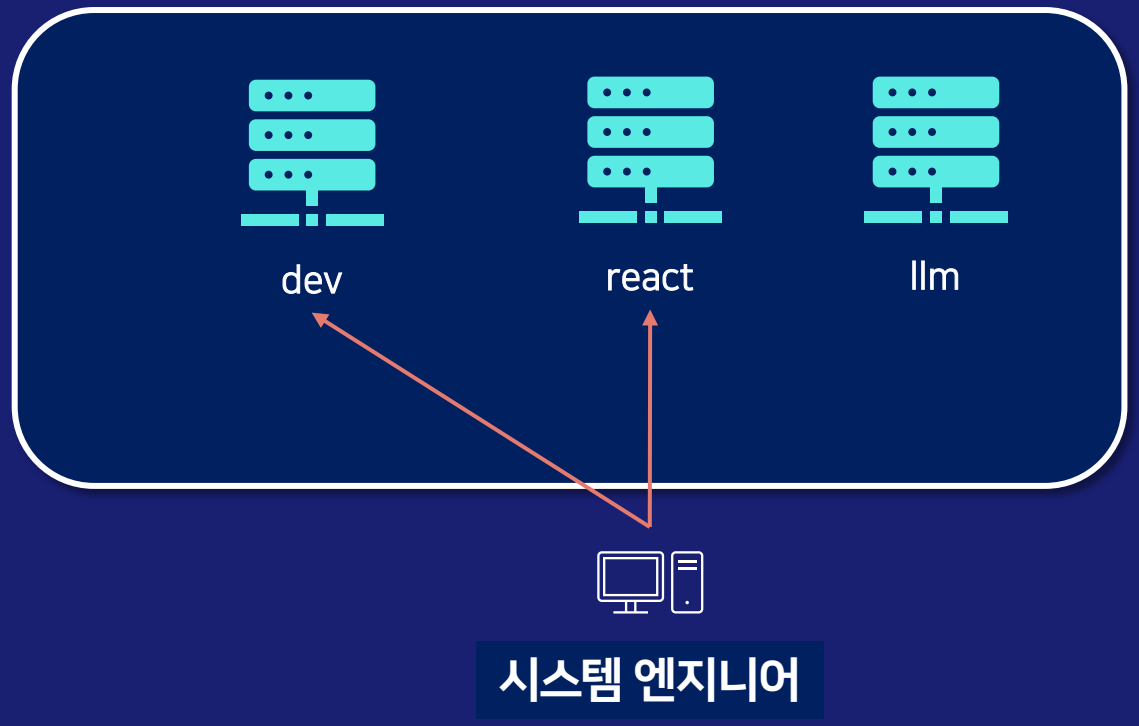
LAB

랩 및 교육대상

React For infra

이 과정은 인프라 엔지니어가 간단한 개발지식 기반으로 React활용하여 실무에서 어떠한 방식으로 적용이 가능한지 다룹니다.

랩 다이어그램



랩 정보

대시보드: `https://vlab.dustbox.kr`

웹 콘솔: `https://vconsole.vlab.dustbox.kr`

접근 시 사용 가능한 계정은 다음과 같음.

- 사용자 계정: `uni1~20`
- 비밀번호: `openworld`
- 도메인: `university`
- Vconsole 콘솔정보: `bastion`

React 소개

React와 기존 자바스크립트 차이

NodeJS 그리고, React 관계

왜 시스템/인프라 엔지니어가 알아야 하는가?

React는 무엇인가?

소개 및 이해하기

ReactJS는 무엇인가?

ReactJS(이하 React)는 **기존 자바 스크립트 기반으로 UI(User Interface)개발을 수월**하게 하기 위해서 함수 기반으로 만든 라이브러리. React는 Meta(구 Facebook)의 엔지니어 Jordan Walke가 개발하였으며, "UI는 상태(State)의 함수이다(UI = f(state))" 라는 개념을 기반으로 설계되었다.React를 사용 시, 최대 장점은 보통 다음과 같다.

1. 기본적인 JavaScript 문법을 이해하면 비교적 빠르게 UI 개발을 시작
2. jQuery와 같은 직접적인 DOM 조작 방식보다 **컴포넌트 기반 구조**를 사용하여 유지보수와 확장이 용이
3. Virtual DOM을 사용하여 실제 DOM 변경을 최소화함으로써 효율적인 UI 업데이트가 가능

```
function App() {
  return <h1>Hello React</h1>;
}
```

DUSTBOX

React와 JavaScript 관계

JavaScript는 이미 알고 있다는 조건으로 설명을 진행하면 다음과 같다.

- 웹 브라우저에서 실행되는 **프로그래밍 언어**
- 웹 페이지의 **동적 기능 구현**에 사용
- HTML, CSS와 함께 **웹의 기본 기술 스택**
- 브라우저 및 서버(Node.js)에서도 실행 가능

자바 스크립트의 주요 목적인 정적인 HTML에서 동적인 웹 애플리케이션의 로직을 작성하는 확장 언어.

```
document.getElementById("title").innerText = "Hello World";
```

ReactJS vs JavaScript

다음과 같이 목적의 차이가 있다.

구분	JavaScript	React
종류	프로그래밍 언어	JavaScript UI 라이브러리
목적	웹 로직 구현	사용자 인터페이스 개발
실행 환경	브라우저 / Node.js	JavaScript 위에서 동작
개발 방식	DOM 직접 제어	컴포넌트 기반 UI

React에서 데이터 핸들링

React는 기본적으로 UI는 f(state) UI는 상태(state)의 함수이다. 말로 표현하면 어렵지만, 아래와 같이 코드로 표현이 가능하다.

```
# 버튼 누르기 전
state = { count: 1 }
UI → "1"

# 버튼 누른 후
state = { count: 2 }
UI → "2"
```

React의 데이터 구조 그리고 디렉터리

컴포넌트 단위로 UI를 생성 및 구성한다. 모든 구성은 디렉터리 형태로 프레임워크가 구성이 된다.

```
App
 ├ Header
 ├ Menu
 ├ Content
 └ Footer
function Header() {
  return <h1>Hello React</h1>;
}
```

React에서 Virtual DOM?

모든 브라우저는 DOM(Document Object Model)이 있다. 웹 브라우저는 HTML를 문서를 읽고 프로그램을 제어할 수 있도록 만든 객체 구조이다. React는 JavaScript에서 일일이 구성하는 DOM 영역을 쉽게 다루도록 라이브러리 수준에서 제어한다.

- HTML = 문서
- DOM = 프로그램이 다룰 수 있는 객체 구조

```
HTML 문서
    ↓
브라우저 파싱
    ↓
DOM Tree 생성
```

React는 DOM를 이렇게 개선하였다

React는 DOM영역을 VirtualDOM으로 다시 재-선언 구조로 더 빠르고 관리가 편하다.

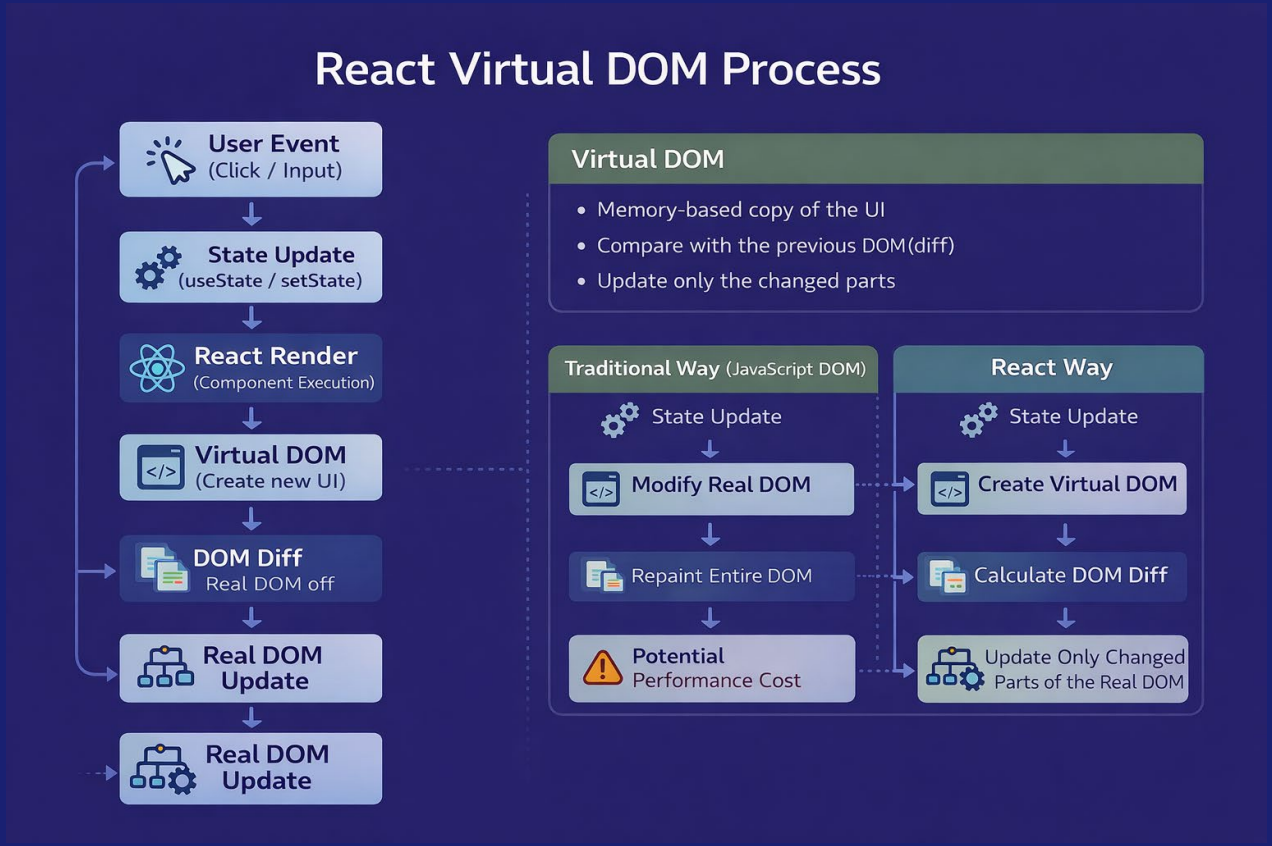
```
Real DOM
    ↑
Virtual DOM
```

동작 방식은 다음과 같다.

1. 상태 변경
2. Virtual DOM 생성
3. 이전 DOM과 비교(diff)
4. 변경된 부분만 실제 DOM 수정

DUSTBOX

Virtual DOM Process



React.JS 코드 예제 및 설명

```
import React from "react";

function App() {
  return (
    <div>
      <h1>Hello React</h1>
      <p>This is my first app</p>
    </div>
  );
}

export default App;
```

NodeJS 코드 예제 및 설명

```
app.post("/api/login", (req, res) => {
  if(req.body.user === "tang"){
     res.json({result:"ok"})
  }
})
```

React와 NodeJS 코드 및 역할 차이 1

구분	React.js 코드	Node.js 코드
역할	Frontend (UI)	Backend (API 서버)
실행 위치	웹 브라우저	서버
목적	사용자 화면 생성	클라이언트 요청 처리
코드 구조	컴포넌트 기반 UI	REST API 처리
주요 기능	HTML 렌더링	데이터 처리 및 응답
사용자와의 관계	사용자 인터페이스 제공	클라이언트 요청 처리
사용 기술	JSX, Component	Express, HTTP
예제 동작	화면에 Hello React 출력	`/api/login` 요청 처리

DUSTBOX

React와 NodeJS 코드 및 역할 차이 2

항목	React 코드	Node.js 코드
기본 구조	Component	API Endpoint
함수 형태	`function App()`	`app.post()`
출력 방식	HTML 렌더링	JSON 응답
사용자 입력 처리	이벤트 / 상태관리	HTTP Request
항목	React 코드	Node.js 코드

DUSTBOX

하지만!

이 교육에서는 NodeJS 기반으로 백 엔드를 구성하지 않습니다. :o

이전보다 Jquery 덜 사용하는 이유 1

React와 Jquery의 기능 차이는 다음과 같다. jQuery는 DOM 조작, 이벤트 처리, AJAX 등을 쉽게 사용하도록 만든 JavaScript 라이브러리이다.

```
document.getElementById("title")
```

```
$("#title")
```

2010년도 까지는 대다수 웹 사이트는 Jquery 기반으로 UI/UX에 많은 적용하였다. 하지만, Jquery에는 다음과 같은 단점이 있다.

- 여전히 DOM를 직접 조작
- JS/UI 상태를 확실히 구별하기가 어려움

이전보다 Jquery 덜 사용하는 이유 2

HTML/CSS/JavaScript가 각각 분리된 구조로 관리되기 때문에 대규모 프로젝트에서 코드 추적 및 유지보수가 어려워질 수 있다.
심지어 개발 도중 사용하지 않는 Jquery 함수나 혹은 선언이 몇 년 내내 계속 유지가 되는 경우가 있다. 이러한 상황이 발생하는 이유는 다음과 같다.

프로젝트 파일은 다음과 같이 보통 구성 및 유지한다.

```
index.js
main.js
utils.js
event.js
dom.js
```

이전보다 Jquery 덜 사용하는 이유 3

그리고 위의 코드에는 대다수가 아래와 같은 형식으로 선언 및 구성이 되어 있다.

```
DOM 수정
DOM 수정
DOM 수정
```

이와 같이 구성이 되면, 코드 유지보수가 점점 어려워지며 최종적으로 이벤트 관련된 네이밍 문제도 발생한다.

```
$("#btn").click(...)
$("#menu").hover(...)
$("#form").submit(...)
```

정리

NodeJS 그리고, React 관계

Front/Backend

NodeJS?

NodeJS는 이미 아시는 내용이지만, 간단하게 JavaScript 기반으로 동작하는 애플리케이션 서버이다. 자바 진영과 비교하면 Servlet, Tomcat이와 같은 구성이다.

- Servlet = Java 웹 프로그램 규격(API)
- Tomcat = Servlet 실행 서버(Web Container)
- Node.js = JavaScript 실행 런타임 + 웹 서버 가능

NodeJS I/O의 코어 구성원

Node.js는 다음과 같은 구조로 이해할 수 있다.

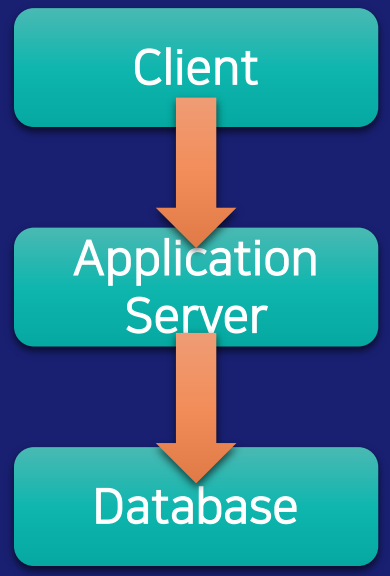
Google Chrome V8 Engine

- JavaScript Runtime
- Event-driven Architecture

= Node.js

3 Tiers Architecture

3 Tiers 기반으로 적용 시, 다음과 같이 NodeJS와 React 적용이 가능하다.

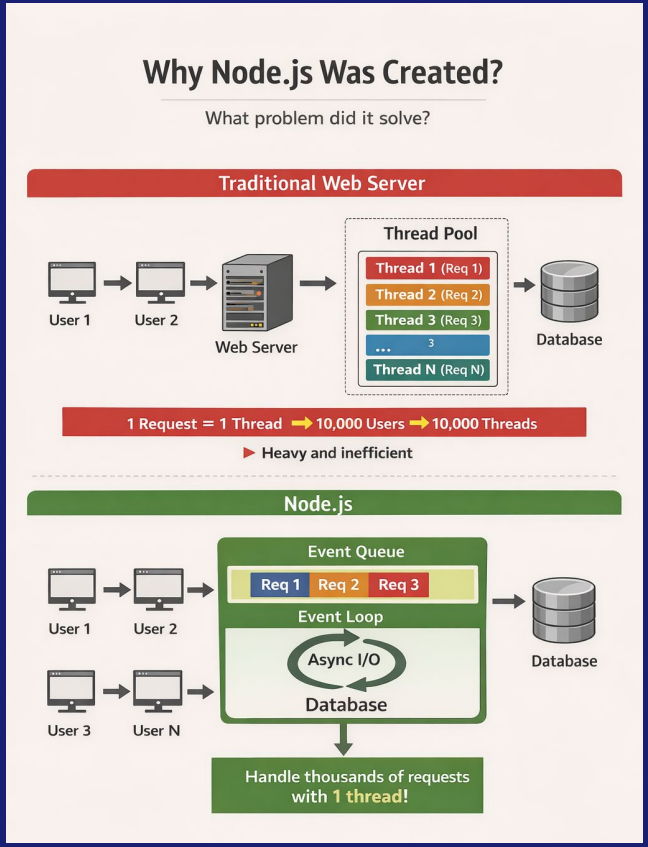


Frontend Tier
- React (Browser UI)

Application Tier
- Node.js Server(Event-driven architecture)

Data Tier
- Database

NodeJS를 사용하는 이유



DUSTBOX

2026-03-19

NodeJS와 이전 구성 비교

기존 톰캣 혹은 미들웨어는 다음과 같은 구조를 가지고 있다.

- Thread Server (Tomcat)
- Request → Thread → Response

하지만, NodeJS는 다음과 같은 방법으로 처리한다.

- Event-driven Server (Node.js)
- Request → Event Queue → Event Loop → Async I/O

NodeJS와 React 적용 대상

현재는 JavaScript만 사용하여도 웹 서비스의 풀 스택 구성이 가능하다. 하지만, NodeJS는 모든 영역에 최선은 아니다.

NodeJS는 보통 다음과 같은 영역에 최적이다.

1. API 서버
2. 실시간 채팅
3. Streaming
4. WebSocket
5. Microservice

NodeJS를 많이 사용하는 대표적인 서비스는 Netflix/Uber/Paypal이 대표적이다.

NodeJS 단점

NodeJS는 자바스크립트 기반으로 되어 있어서 빠르게 개발이 가능하다는 장점이 있다. 하지만, 치명적인(?) 단점을 다음과 같이 가지고 있다.

1. NodeJS 설계 실수
2. 보안구조 문제
3. 패키지 구조 문제

이를 해결하기 위해서 Ryan Dalh은 새로운 언어인 Deno를 구성하기 시작했다.

https://deno.com/deploy/sandbox

NodeJS와 React의 관계 정리

React는 사용자 **인터페이스(UI)를 구성하기 위한 프론트엔드 라이브러리.**

NodeJS는 서버에서 동작하는 백엔드 런타임 환경. 두 기술은 역할과 실행 환경이 서로 다르며, 반드시 함께 사용해야 하는 관계는 아니다.

React는 HTTP 기반 API를 통해 다양한 백엔드 시스템과 연동할 수 있다.

예를 들어 **Go, Python, Java와 같은 백엔드와도 문제없이 통합이 가능**하다. 특히 인프라 엔지니어 환경에서는 자동화, 배포, 운영 효율을 고려하여 NodeJS 대신 다른 백엔드를 선택하는 경우도 많다.

따라서 React와 NodeJS는 필수적인 조합이 아니라, 상황에 따라 선택 가능한 독립적인 기술이다.

왜 시스템/인프라 엔지니어가 React를 알아야 하는가?

현재 인프라 엔지니어의 역량

먼저, 시스템 엔지니어는?

Computer System Engineer

컴퓨터 시스템의 **설계, 구축, 통합, 운영 및 최적화**를 담당하는 엔지니어

주요 역할

System Architecture
- 분산 시스템 및 서비스 구조 설계

Infrastructure Engineering
- 서버, 네트워크, 스토리지, 가상화 환경 구축

Platform Engineering
- Kubernetes, Cloud Platform 구축 및 운영

Automation Engineering
- IaC, CI/CD 기반 자동화

Reliability Engineering
- 모니터링, 장애 대응, 고가용성 설계

Cloud/Kubernetes/IaC
https://www.cncf.io/reports/cncf-annual-survey/
https://training.linuxfoundation.org/resources/

개발 및 스크립팅 (DevOps 트렌드)
https://survey.stackoverflow.co/2024/
https://github.blog/2023-06-08-developer-productivity-ai/

Observability/SRE
https://sre.google/sre-book/table-of-contents/
https://sre.google/resources/practices-and-processes/

계속

DUSTBOX

현대 시스템 엔지니어의 확장된 역할

Platform Engineering

- Kubernetes
- Container Platform
- Cloud Platform

Reliability Engineering

- Monitoring
- Observability
- High Availability

Platform Tooling

- Web 기반 관리 도구
- API 기반 시스템 관리

최근 시스템 엔지니어 핵심 역량
Operating System · Networking · Distributed System · Automation

요즘 인프라 관리 도구

현재 많이 사용하는 인프라 도구들의 대다수는 React혹은 JS기반으로 다루는 DOM 형식을 가지고 있다. 아래 대표적인 도구들이 **React**혹은 SPA(Single Page Application)구조를 가지고 있다.

- Kubernetes Dashboard
- Harbor Container Registry
- Grafana Dashboard
- Rancher

위의 도구들이 전부 React를 사용하지는 않지만, 기본적으로 JS기반으로 사용하는 라이브러리를 사용하고 있다. SPA를 구조 및 구성을 알아야지 어떠한 방식으로 시스템 관리가 되는지 API가 호출이 되는지 확인이 가능하다.

현재는 API 중심

1990년도에는 로컬 애플리케이션이 API 중심이었고, 웹은 HTML 및 JS 기반으로 구성된 여러 개의 페이지 및 라이브러리 기반으로 구성이 되었다.

2000년도로 넘어오면서 대다수 인프라 서비스 및 애플리케이션 서비스가 웹 중심으로 넘어오면서 웹 및 모바일 애플리케이션도 API로 전환이 시작이 되었다.

앞으로 시스템 엔지니어도 운영 시스템 구축 및 운영을 위해서는 React와 같은 SPA 라이브러리를 사용하여 운영 애플리케이션 구성 및 작성을 매우 권장하고 있다.

왜 인프라 엔지니어가 React를 알아야 하는가

과거 인프라 엔지니어의 역할

- 서버 설치
- 네트워크 구성
- 서비스 운영 및 관리

현재 인프라 환경 변화

- Cloud/Kubernetes/Container Platform 등장
- 인프라 엔지니어 → Platform Engineer

플랫폼 운영의 특징

- CLI 중심 운영 → Web 기반 관리 도구 증가
- 대부분 API + Web UI 구조

DUSTBOX

요즘 인프라 관리 도구

현재 많이 사용하는 인프라 도구들의 대다수는 React 혹은 JS 기반으로 DOM 형식을 가지고 있다.

대표적인 도구

1. Kubernetes Dashboard
2. Harbor Container Registry
3. Grafana Dashboard
4. Rancher

SPA(Single Page Application) 구조를 기반으로 동작

결론, 현재는 API 중심

1990년대

- 로컬 애플리케이션 중심

2000년대

- 웹 기반 애플리케이션 증가

현재

- API 중심 서비스 구조
- React와 같은 SPA 기반 UI 사용 증가

개발환경 구성

오픈스택 가상머신

환경설명

랩 환경은 다음처럼 구성이 된다.

1. VDI 환경(가상머신)
2. DEV 가상머신
3. TEST 가상머신

각 모든 결과물은 컨테이너 기반으로 배포가 된다. 개발 환경 또한 VDI기반으로 테스트 및 컴파일 한다.

NodeJS 및 NPM 설치

아래 명령어로 개발서버(dev)에 NodeJS, NPM를 설치한다.

```
dnf module enable nodejs:20 -y
dnf install nodejs -y
node -v
v16.20.2
npm -v
8.19.4
```

React 프로젝트 생성

React프로젝트는 여러 방법으로 생성이 가능하다.

1. Vite 기반으로 생성
2. Create React App

2번은 구형 방식이기 때문에 현재는 Vite 기반으로 생성을 권장한다. 다음 조건으로 React 프로젝트를 생성한다.

- `framework: React`
- `variant: TypeScript`
- `Vite 8 beta: No`
- `npm and start now: No`

프로젝트 구성

아래 명령어로 프로젝트를 구성한다.

```
npm create vite@latest my-react-app
|
◇  Target directory "my-react-app" is not empty. Please choose how to
proceed:
|  Remove existing files and continue
|
◇  Select a framework:
|  React
|
```

계속

프로젝트 구성

```
◇  Select a variant:
│  TypeScript
│
◇  Use Vite 8 beta (Experimental)?:
│  No
│
◇  Install with npm and start now?
│  No
│
◇  Scaffolding project in /root/my-react-app...
│
└  Done. Now run:
```

계속

프로젝트 구성

```
  cd my-react-app
  npm install
  npm run dev
cd my-react-app/
npm run dev -- --host 0.0.0.0 --port 8801
```

기본 컴포넌트 학습

랩 구성

간단한 테스트

문법 및 기능 확인

기본 문법 학습

기본 학습을 진행하기 위해서 다음 명령어로 react-tutorial/라는 디렉터리를 생성한다. 조건은 앞에서 하였던 내용과 동일하게 다음과 같이 실행한다.

1. React
2. TypeScript

이 조건으로 **react-tutorial/** 프로젝트를 생성한다. 실행 시, 외부에서 접근하기 위해서 다음과 같이 포트를 노출한다.

```
cd react-tutorial/
npm run dev -- --host 0.0.0.0 --port 8801
```

JSX?

React는 기본적으로 JSX(JavaScript XML)라는 형태를 가지고 있다. HTML 태그와 동일하게 **큰 따옴표("")**를 사용하지 않는다. 그러하면서 자바스크립트와 같은 구조를 동시에 가지고 있다.

JSX는 기본적으로 **JavaScript+XML**이 합쳐진 구조. 자바스크립트의 확장된 문법. React에서는 JSX사용을 권고하고 있지만, 반드시 JSX사용하라는 강제사항도 없다.

다만, JSX를 사용하면 좀 더 애플리케이션 속도 및 가독성이 높아지기 때문에, 사용을 적극적으로 권장한다.

JavaScript in React? 1

가능하다. 하지만, 다음 예제는 다음과 같은 동작 조건을 가지고 있다. 이 교육에서는 JavaScript를 최소화 한다.

- React JSX안에서 JavaScript 표현식 {} 사용
- 변수+함수+조건문을 React에서 사용하는 방법

```
import React from "react";

function Example() {
  // JavaScript 변수
  const userName = "Tang";
  // JavaScript 함수
  function getScore() {
    return 85;
  }
```

자바스크립트 함수 선언

계속

JavaScript in React? 2

```
  // JavaScript 조건
  const score = getScore();
  const result = score ≥ 60 ? "Pass" : "Fail";

  return (
    <div>
      <h2>React + JavaScript Example</h2>
      <p>User : {userName}</p>
      <p>Score : {score}</p>
      <p>Result : {result}</p>
    </div>
  );
}
export default Example;
```

구성이 올바르게 되었는지 확인 1

React는 기본적으로 **프론트엔드 라이브러리 및 도구**이기 때문에 사용방법만 이해하면 사용이 매우 간단하다.

웹 프론트 메인 페이지는 `src/App.tsx`에서 실행이 된다. 해당 코드를 수정하면 React는 HotReLoad(HRL)를 지원하기 때문에 바로 웹 페이지에서 변경이 된다.

올바르게 구성이 되었으면 다음과 같이 간단하게 코드를 변경한다. 기존에 있는 내용은 전부 제거하고 아래와 같이 입력한다.

```
vi src/App.tsx
type UserProps = {
  name: string
  role: string
}
```

변수 선언

계속

구성이 올바르게 되었는지 확인 2

```
function User({name, role}:UserProps){
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  )
}
```

인자 값 전달

변수 출력

계속

구성이 올바르게 되었는지 확인 3

```
export default function App(){
  return(
    <div>
      <User name="국현" role="Infra Engineer"/>
      <User name="아름이" role="Assistant"/>
    </div>
  )
}
```

HTML/CSS 출력 부분

예상 결과

문제가 없으면 다음과 같이 화면에 출력이 된다. npm 서버가 실행중인 터미널은 절대 닫으면 안된다.

React 디렉터리 설명 1

기본적으로 npm 명령어로 다음과 같이 구성이 된다.

```
react-tutorial
│
├── public
├── src
├── node_modules
├── package.json
```

디렉터리는 npm이 자동으로 생성한다.
수동으로 생성 및 관리는 추천하지 않는다.

계속

React 디렉터리 설명 2

프로젝트 루트 디렉터리는 `react-tutorial/`이다. 여기에는 다음과 같은 정보가 있다.

`react-tutorial/`

1. 설치파일
2. 패키지정보
3. 소스코드
4. 정적파일

계속

React 디렉터리 설명 3

package.json은 React 프로젝트에서 사용하는 패키지 관리 파일이다. 이 파일에는 다음과 같은 내용이 있다.

1. 프로젝트 정보
2. 사용하는 라이브러리 목록
3. 스크립트 명령어
4. 프로그램 버전 정보

```
# package.json
npm run dev
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

scripts에는 dev, build, preview 실행 시 사용할 명령어가 정의되어 있다.

계속

React 디렉터리 설명 4

프로젝트에서 사용하는 **모든 라이브러리 패키지**가 설치되는 디렉터리.

이 디렉터리의 내용은 npm install 명령어 실행 시 자동으로 생성이 된다. GIT 서버에 업로드 시, .gitignore에 보통 추가한다.

```
ls react-tutorial/
react
react-dom
vite
eslint
typescript
```

계속

React 디렉터리 설명 5

웹 서버에서 **그대로 제공되는 정적 파일**이 위치하는 디렉터리이다.

이 디렉터리의 내용은 **`npm install`** 명령어 실행 시 자동으로 생성이 된다. 이 내용은 GIT 서버에 업로드 시, .gitignore에 보통 추가한다.

```
ls public
favicon
logo
icons
images
```

이 안에는 공통으로 생성 및 제공되는 파일

계속

React 디렉터리 구성 설명

src/ 디렉터리는 React 애플리케이션의 **실제 코드가 위치하는 디렉터리**이다. 이 내부에는 실제로 React가 실행 시 사용하는 코드들이 위치.

```
ls src/
components
css
assets/
logic
```

실제로 사용하는 소스코드 및 css가 포함이 되어 있다.

React 렌더링 시작 구조 – main.tsx

React 애플리케이션의 **시작점(entry point)**이다. 여기서 React 앱이 브라우저에 렌더링 된다.

```
~~import React from 'react'~~
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

해당 부분은 더 이상 선언하지 않아도 된다.

react 라이브러리 불러오기, 애플리케이션 디렉터리 선언.

계속

DUSTBOX

React 렌더링 구조 – main.tsx

main.tsx는 React 애플리케이션의 시작 지점으로, App 컴포넌트를 브라우저의 root DOM에 렌더링한다.

```
ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
index.html → main.tsx → App.tsx → Components (UI 구성)
```

React와 ReactDOM을 불러오고,렌더링할 App 컴포넌트를 가져온다.

React 루트 컴포넌트 – App.tsx

App.tsx는 React 애플리케이션의 루트 컴포넌트로,전체 UI 구조를 정의하고 하위 컴포넌트를 구성하는 역할을 한다.

```
function App() {
  return (
    <div>
      <h1>Hello React</h1>
    </div>
  )
}
export default App
```

실행 시, 웹 페이지에 Hello React라고 출력. JSX에서는 가급적이면 처럼 요소(elements) 기반으로 구성한다.

이 컴포넌트를 다른 파일에서 사용할 수 있도록 export 한다

DUSTBOX

React 렌더링 시작 – index.html

index.html은 React 애플리케이션이 브라우저에서 실행될 때 사용되는 기본 HTML 파일이다. index.html은 Vite가 제공하는 기본 HTML 파일로,이 파일의 root 영역에 React 앱이 렌더링 된다.

일반적으로 자주 수정하지는 않지만,필요에 따라 title, meta 정보 등을 수정할 수 있다.

```
<div id="root"></div>
```

Vite 설정 파일 – vite.config.ts

vite.config.ts는 Vite 개발 서버와 빌드 동작을 설정하는 파일로,포트, 플러그인, 빌드 옵션 등을 정의한다

1. 개발 서버 설정 (포트, 호스트)
2. 플러그인 설정 (React 등)
3. 빌드 설정 (번들링 옵션)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    host: true
  }
})
```

개발 서버 실행 시 동작 방식과 환경을 제어하기 위해 사용한다.

DUSTBOX

Vite의 역할 (개발 서버 + 빌드 도구)

Vite는 React 같은 프론트엔드 애플리케이션을 개발하고 빌드하기 위한 개발 서버+빌드 도구다. 실 개발 시, 손대는 경우는 거의 없다.

```
Vite = "React 개발 서버" + "빌드 시스템"
```

브라우저는 JavaScript만 실행할 수 있기 때문에,TypeScript와 JSX 코드는 그대로 실행할 수 없다. 이러한 이유로 Vite는 각각 코드를 사용이 가능하도록 해석 및 컴파일 작업을 수행한다.

```
React Code(ts, jsx)
      |
      ▼
   Vite
      |
      ▼
Browser JS(JavaScript)
```

DUSTBOX

TypeScript 설정 파일 (tsconfig)

TypeScript 컴파일 설정 파일이다. 다음과 같은 내용들이 포함이 되어있다.

1. 타입 체크 방식 설정
2. JavaScript 변환 방식 설정
3. import/export 동작 설정

```
tsconfig.json       → 전체 설정 (기본)
tsconfig.app.json   → 브라우저용 코드 설정
tsconfig.node.json → Node.js 환경 설정
```

기본 컴포넌트 학습

문법

프레임워크 소개 1

React(라이브러리)를 사용 시, 다양한 JavaScript기반 프레임 워크는 다음과 같다. 어떠한 프레임워크를 사용하느냐 따라서 사용 방법 및 문법이 조금씩 다를 수 있다.

프레임워크	언어	특징
React	JavaScript / TypeScript	컴포넌트 기반 UI 라이브러리
Vue.js	JavaScript	직권적 문법, 빠른 학습
Angular	TypeScript	구조화된 대규모 앱 개발
Svelte	JavaScript	빌드 시 컴파일
Next.js	React 기반	SSR/SEO
Nuxt.js	Vue 기반	SSR 지원
Remix	React 기반	서버 중심 아키텍처

프레임워크 소개 2

백엔드는 React 영역과 다르지만, 서로 호환 및 사용이 가능한 영역은 다음과 같다.

프레임워크	언어	특징
Express.js	Node.js	최소한의 기능, 자유로운 구조
NestJS	Node.js/TypeScript	구조화된 서버, Angular 스타일의 구조화된 서버
Spring Boot	Java	엔터프라이즈
Django	Python	풀스택 프레임워크
FastAPI	Python	매우 빠른 API
Flask	Python	경량
Gin	Go	Go 기반 프레임워크(고성능)
Echo	Go	Go REST API

DUSTBOX

변수와 State의 차이

React에서는 일반 변수도 사용할 수 있지만,UI와 연결된 값은 state로 관리해야 화면이 업데이트된다.

```
## JavaScript
var name = "Tang"
var count = 0
let count = 0
count++

## React
const name = "Tang"
const [count, setCount] = useState(0)
```

값은 변경되지만, 화면은 자동으로 업데이트 되지 않는다.

count → 상태 값
setCount → 상태를 변경하는 함수

React 상태 변경 예제

다음과 같이 변수를 선언한다. 네이티브 JavaScript 코드 및 함수 사용이 가능하다.

```
function Counter(){
  const [count, setCount] = useState(0)
  function increase(){
    setCount(count + 1)
  }
  return (
    <div>
      <p>{count}</p>
      <button onClick={increase}>+</button>
    </div>
  )
}
export default Counter
```

React에서도 JavaScript를 그대로 사용할 수 있지만,화면과 연결된 값은 state로 관리해야 한다.

일반 변수 출력 예제

React에서는 일반 변수도 JSX에서 출력할 수 있다.

```
function Example1() {
  const name = "Tang"
  return (
    <div>
      <h2>이름: {name}</h2>
    </div>
  )
}
export default Example1
```

일반 변수는 값이 변경되어도 화면이 자동으로 업데이트되지 않는다.

JavaScript 값 → JSX 출력

문자열뿐만 아니라 숫자도 JSX에서 출력할 수 있다.

```
function Example2() {
  const age = 45
  return (
    <div>
      <h2>나이: {age}</h2>
      <h2>5년후 나이: {age+5}</h2>
    </div>
  )
}
export default Example2
```

JS 값 조합 → JSX 출력

JSX에서는 여러 JavaScript 값을 조합하여 하나의 결과로 출력할 수 있다.

```
function Example3() {
  const firstName = "Tang"
  const lastName = "Choi"

  return (
    <div>
      <h2>{firstName} {lastName}</h2>
      <h2>{firstName + " " + lastName}</h2>
    </div>
  )
}
export default Example3
```

중괄호 안은 HTML이 아니라 JavaScript입니다

JS 값 조합 → JSX 출력

JavaScript 연산 결과를 JSX에서 출력할 수 있다.

```
function Example4() {
  const a = 10
  const b = 20
  const result = a + b
  return (
    <div>
      <h2>결과: {result}</h2>
      <h2>직접 계산: {a + b}</h2>
    </div>
  )
}
export default Example4
```

중괄호 안에서는 계산도 바로 됩니다

조건 렌더링 (삼항 연산자)

JSX에서는 JavaScript 조건문을 사용해 출력 내용을 변경할 수 있다.

```
function Example5() {
  const isAdmin = true
  return (
    <div>
      <h2>권한: {isAdmin ? "관리자입니다" : "사용자입니다"}</h2>
    </div>
  )
}
export default Example5
```

JSX 안에서도 if는 못 쓰지만, 조건 연산자는 사용할 수 있습니다

배열 → JSX 출력

배열은 다음과 같이 출력이 가능하다. 아래 방식은 사용은 가능하나, 권고하지는 않는다.

```
function Example6() {
  const fruits = ["apple","banana","orange"]
  return (
    <div>
      <h2>{fruits[0]}</h2>
      <h2>{fruits[1]}</h2>
      <h2>{fruits[2]}</h2>
    </div>
  )
}
export default Example6
```

배열 → JSX 출력

위의 내용을 map()함수를 사용하면, 다음과 같이 출력이 가능하다.

```
function Example7() {
  const fruits = ["apple","banana","orange"]
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  )
}
export default Example7
```

이와 같은 map 형태로 사용을 권장합니다.

객체 → JSX 출력

객체의 프로퍼티에 접근하여 JSX에 출력할 수 있다.

```
function Example8() {
  const user = {
    name: "Tang",
    age: 45
  }
  return (
    <div>
      <h2>이름: {user.name}</h2>
      <h2>나이: {user.age}</h2>
    </div>
  )
}
export default Example8
```

객체의 프로퍼티에 접근하여 JSX에 출력할 수 있다.

참고: JavaScript에서는 딕셔너리는 없음. JS에서는 전부 객체(Object)

상태(State) 변경과 렌더링

state 값이 변경되면 화면도 함께 업데이트된다.

```
import { useState } from "react"
function Example9() {
  const [count, setCount] = useState(0)
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() ⇒ setCount(count + 1)}>
        증가
      </button>
    </div>
  )
}
export default Example9
```

state가 바뀌면 화면이 다시 그려집니다

입력 값과 state 연결

입력 값을 state에 저장하고 화면에 반영할 수 있다.

```
import { useState } from "react"
function Example10() {
  const [name, setName] = useState("")
  return (
    <div>
      <input
        type="text"
        placeholder="이름을 입력하세요"
        onChange={(e) => setName(e.target.value)}
      />
      <h2>{name}</h2>
    </div>
  )
}
export default Example10
```

연습문제

변수 미니 프로젝트

설명

앞에서 학습한 변수 내용을 가지고 아래 프로젝트 코드를 완성한다. 위의 내용을 참조해서 진행한다.

프로젝트 이름

- **학생 정보 카드 대시보드**

프로젝트 목적

- React에서 자주 쓰는 변수 문법을 한 번에 연습한다.
- const 변수
- 객체 변수
- 배열 변수
- 조건부 렌더링
- map() 출력
- useState() 상태값 변경

동작화면

화면에 이런 내용이 나온다.

- 학생 이름
- 나이
- 수강 과목 목록
- 점수
- 합격/불합격
- 버튼 클릭 시 점수 증가

코드

```
import { useState } from "react"

function App() {
  const academyName = "React 변수 실습"
  const teacher = "국현"

  const student = {
    name: "Tang",
    age: 20,
    job: "Frontend Student"
  }
```

코드

```
const subjects = ["HTML", "CSS", "JavaScript", "React"]
const [score, setScore] = useState(65)
const isPass = score ≥ 70
const increaseScore = () ⇒ {
  setScore(score + 5)
}

const resetScore = () ⇒ {
  setScore(65)
}
```

코드

```
return (
   <div style={{ padding: "30px", fontFamily: "sans-serif" }}>
     <h1>{academyName}</h1>
     <h2>강사: {teacher}</h2>

     <hr />

     <h2>학생 정보</h2>
     <p>이름: {student.name}</p>
     <p>나이: {student.age}</p>
     <p>직무: {student.job}</p>

     <hr />
```

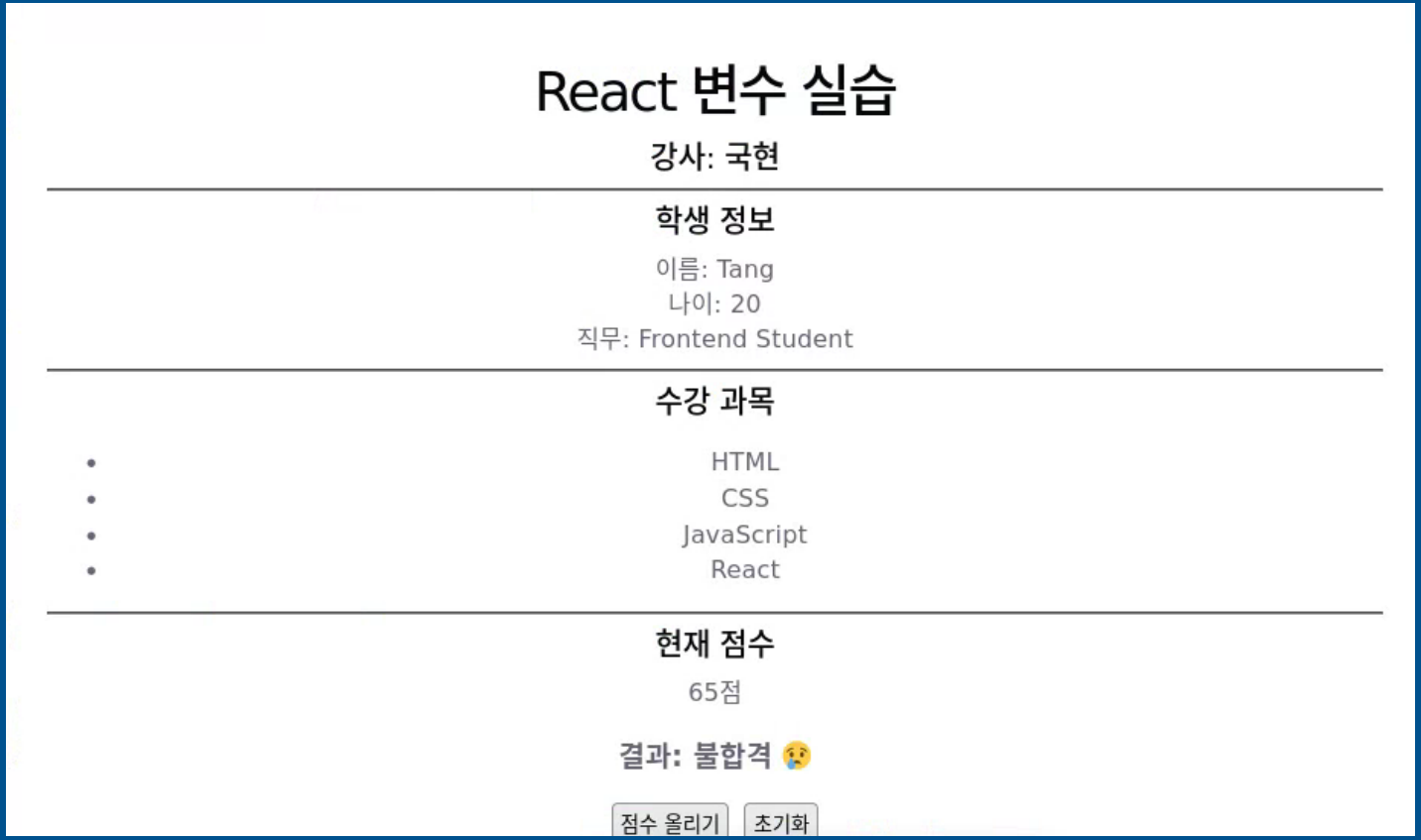
코드

```
<h2>수강 과목</h2>
    <ul>
      {subjects.map((subject, index) => (
        <li key={index}>{subject}</li>
      ))}
    </ul>

    <hr />

    <h2>현재 점수</h2>
    <p>{score}점</p>
```

코드

```
 <h3>
        결과: {isPass ? "합격 🎉" : "불합격 😥"}
      </h3>
      <button onClick={increaseScore} style={{ marginRight: "10px" }}>
        점수 올리기
      </button>
      <button onClick={resetScore}>
        초기화
      </button>
    </div>
  )
}
export default App
```

예상 결과 화면


React 변수 실습
강사: 국현
학생 정보
이름: Tang
나이: 20
직무: Frontend Student
수강 과목
HTML
CSS
JavaScript
React
현재 점수
65점
결과: 불합격 😢
점수 올리기
초기화

Properties, Props 적용

UI 속성 적용

Props(Properties)설명

React에서 **부모 컴포넌트 → 자식 컴포넌트로 데이터를 전달하는 방식.**

핵심 특징은 다음과 같다.

- props = 부모가 자식에게 전달하는 데이터
- state = 컴포넌트 내부에서 관리하는 데이터

React에서 사용하는 모든 컴포넌트는 함수처럼 동작한다. 즉, 특정 컴포넌트에 **속성(Props, Properties) 변경 후 UI에 적용**한다.

즉, Props는 함수의 **파라미터(Parameter)**이다.

Props 사용 방법

부모가 전달한 props를 받아서 화면에 출력한다.

함수에 전달할 문자열 변수 `UserProps`는 다음과 같이 함수에 전달한다. 함수변수는 `{ name }`으로 `UserProps`에서 불러온 후, `Hello { VARIABLE }`를 변수를 출력한다.

```
type UserProps = {
  name: string
}
```

```
function User({ name }: UserProps) {
  return <h2>Hello {name}</h2>
}
```

Props 사용 방법

다음과 같이 부모/자식 컴포넌트에 적용이 가능하다. 먼저, 부모 컴포넌트를 생성 및 구성한다. DIV를 활용해서 간단하게 레이어를 구성한다.

```
type UserProps = {
  name: string
}

// 부모
<User name="Tang" />

// 자식
function User({ name }: UserProps) {
  return <h2>Hello {name}</h2>
}
```

기본적인 props 예제

부모 컴포넌트가 props를 전달하고, 자식 컴포넌트가 이를 사용한다.

```
export default function App() {
  return (
    <div>
      <User name="Tang" />
      <User name="Infra Engineer" />
    </div>
  )
}

function User({ name }) {
  return <h2>Hello {name}</h2>
}
```

진행 전…

다음과 같이 props랩을 위한 디렉터리 생성 및 프로젝트 파일을 구성한다.

```
npm create vite@latest react-props-lab
Framework: React
Variant: TypeScript
No
cd react-props-lab
npm install
mkdir src/components
touch src/components/Example1UserCard.tsx
touch src/components/Example2ProductCard.tsx
touch src/components/Example3Button.tsx
touch src/components/Example4Profile.tsx
touch src/components/Example5StudentList.tsx
```

components/Example1UserCard.tsx

아래와 같이 prosps를 생성 및 적용한다. 두 개의 strings를 통해서 값을 전달한다.

```
type UserCardProps = {
  name: string
  role: string
}
export default function Example1UserCard({ name, role }: UserCardProps) {
  return (
    <div>
      <h2>이름: {name}</h2>
      <p>역할: {role}</p>
    </div>
  )
}
```

App.tsx

App.tsx에 다음과 같이 작성한다.

```
import Example1UserCard from "./components/Example1UserCard"

export default function App() {
  return (
    <div>
      <h1>Props Example</h1>
      <Example1UserCard name="국현" role="Infra Engineer" />
    </div>
  )
}
```

components/Example2ProductCard.tsx

```
type ProductCardProps = {
  title: string
  price: number
}
export default function Example2ProductCard({ title, price }:
ProductCardProps) {
  return (
    <div>
      <h2>상품명: {title}</h2>
      <p>가격: {price}원</p>
    </div>
  )
}
```

App.tsx

App.tsx에 다음과 같이 작성한다.

```
import Example2ProductCard from "./components/Example2ProductCard"

export default function App() {
  return (
    <div>
      <h1>숫자 Props 예제</h1>
      <Example2ProductCard title="React 교재" price={30000} />
      <Example2ProductCard title="TypeScript 교재" price={45000} />
    </div>
  )
}
```

components/Example3Button.tsx

```
type ButtonProps = {
  label: string
  onClick: () ⇒ void
}

export default function Example3Button({ label, onClick }: ButtonProps) {
  return (
    <div>
      <h2>Example3 - Button</h2>
      <button onClick={onClick}>{label}</button>
    </div>
  )
}
```

App.tsx

App.tsx에 다음과 같이 작성한다.

```
import Example3Button from "./components/Example3Button"
export default function App() {
  const hello = () ⇒ {
    alert("Button Clicked!")
  }
  return (
    <div style={{padding:"20px"}}>
      <h1>Example3 - Function Props</h1>
      <Example3Button label="클릭하세요" onClick={hello}/>
    </div>
  )
}
```

Example4Profile.tsx 1

```
type Profile = {
  name: string
  age: number
  job: string
}

type ProfileProps = {
  user: Profile
}
```

components/Example4Profile.tsx 2

```
export default function Example4Profile({ user }: ProfileProps) {
  return (
    <div>
      <h2>Example4 - Profile</h2>
      <p>이름: {user.name}</p>
      <p>나이: {user.age}</p>
      <p>직업: {user.job}</p>
    </div>
  )
}
```

App.tsx

App.tsx에 다음과 같이 작성한다.

```
import Example4Profile from "./components/Example4Profile"

export default function App() {

  const user = {
    name: "국현",
    age: 44,
    job: "Infra Engineer"
  }
```

계속

App.tsx

```
  return (
    <div style={{padding:"20px"}}>
      <h1>Example4 - Object Props</h1>

      <Example4Profile
        user={user}
      />

    </div>
  )
}
```

components/Example5Profile.tsx 1

```
type StudentListProps = {
  students: string[]
}
export default function Example5StudentList({ students }: StudentListProps)
{
  return (
    <div>
      <h2>Example5 - Student List</h2>
      <ul> {students.map((student, index) => (<li
key={index}>{student}</li>))}
      </ul>
    </div>
  )
}
```

App.tsx

App.tsx에 다음과 같이 작성한다.

```
import Example5StudentList from "./components/Example5StudentList"

export default function App() {

  const students = [
    "Tang",
    "Kim",
    "Lee",
    "Park",
    "Choi"
  ]
```

App.tsx

```
  return (
    <div style={{padding:"20px"}}>
      <h1>Example5 - Array Props</h1>

      <Example5StudentList
        students={students}
      />

    </div>
  )
}
```

페이지 통합하기

지금은 각 애플리케이션 별로, App.tsx에 따로 적용하였다. 한 개 이상의 페이지는 SPA처럼 아래와 같이 구성이 가능하다. 여기서는 compoents/라는 디렉터리를 생성 후, 기능을 몰아 넣는다.

```
src/
├── App.tsx
├── components/
│   ├── Example1UserCard.tsx
│   ├── Example2ProductCard.tsx
│   ├── Example3Button.tsx
│   ├── Example4Profile.tsx
│   └── Example5StudentList.tsx
└── main.tsx
```

페이지 통합하기 1

App.tsx에 다음과 같이 모든 컴포넌트를 등록한다.

```
import Example1UserCard from "./components/Example1UserCard"
import Example2ProductCard from "./components/Example2ProductCard"
import Example3Button from "./components/Example3Button"
import Example4Profile from "./components/Example4Profile"
import Example5StudentList from "./components/Example5StudentList"
export default function App() {
  const user = {
    name: "국현",
    age: 44,
    job: "Infra Engineer",
  }
```

모든 컴포넌트를 불러온다. 하나라도 올바르게 불러오지 않으면 렌더링이 되지 않는다.

페이지 통합하기 2

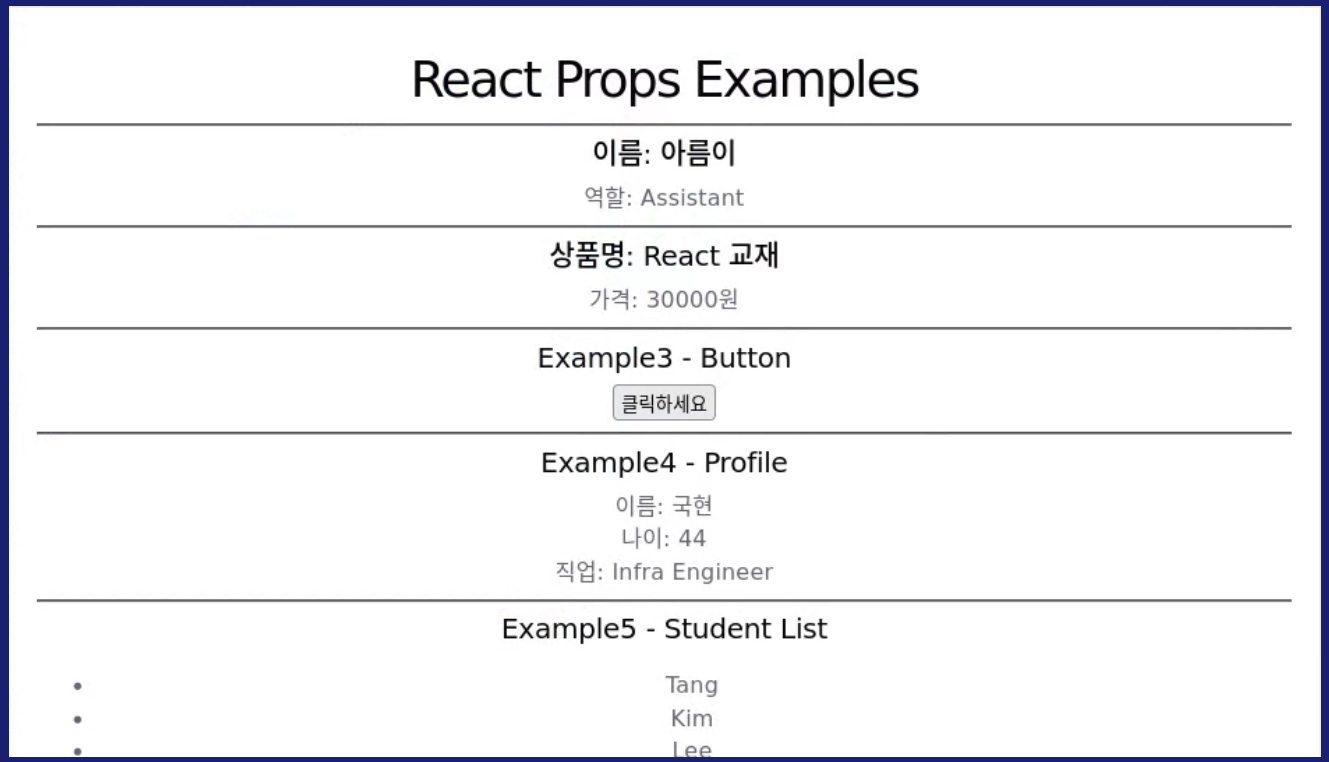
```
const students = ["Tang", "Kim", "Lee", "Park"]
const hello = () => {
  alert("Hello Props!")
}
return (
  <div style={{ padding: "20px", fontFamily: "sans-serif" }}>
    <h1>Props Examples</h1>
    <hr />
    <Example1UserCard name="아름이" role="Assistant" />
    <hr />
```

페이지 통합하기 3

```
      <Example2ProductCard title="React 교재" price={30000} />
      <hr />
      <Example3Button label="클릭하세요" onClick={hello} />
      <hr />
      <Example4Profile user={user} />
      <hr />
      <Example5StudentList students={students} />
    </div>
  )
}
```

예상화면

문제없이 페이지가 통합이 되면 다음과 같이 모든 내용이 출력이 된다.



연습문제

Props 미니 프로젝트

사용자 목록

미니 프로젝트

설명

앞에서 학습한 **props 내용을 가지고** 아래 프로젝트 코드를 완성한다. 위의 내용을 참고해서 진행한다.

프로젝트 이름

- 사용자 정보 카드 대시보드

프로젝트 목적

React에서 자주 사용하는 **props 문법을 한 번에 연습한다.**

- 문자열 props 전달
- 숫자 props 전달
- 객체 props 전달
- 배열 props 전달
- 함수 props 전달
- map() 출력
- 컴포넌트 분리 및 재사용

프로젝트 생성

프로젝트 진행 전, 아래와 같이 라이브러리 및 디렉터리를 준비한다.

```
npm create vite@latest react-props-dashboard
Framework: React
Variant: TypeScript
Install with npm and start now?: No
cd react-props-dashboard
npm install
mkdir -p src/components
touch src/components/UserCard.tsx
touch src/components/UserList.tsx
npm run dev -- --host 0.0.0.0 --port 8801
```

사용자 카드 생성

프로젝트 진행 전, 아래와 같이 라이브러리 및 디렉터리를 준비한다.

```
vi src/components/UserCard.tsx
type UserCardProps = {
  name: string
  role: string
  skill: string
}
```

사용자 카드 생성

```
export default function UserCard({ name, role, skill }: UserCardProps) {
  return (
    <div style={{
      border: "1px solid gray",
      padding: "10px",
      margin: "10px",
      width: "200px"
    }}>
      <h3>{name}</h3>
      <p>{role}</p>
      <p>{skill}</p>
    </div>
  )
}
```

사용자 목록

사용자 정보를 출력하는 카드를 생성 및 구성한다.

```
vi src/components/UserList.tsx
import UserCard from "./UserCard"

type User = {
  name: string
  role: string
  skill: string
}

type UserListProps = {
  users: User[]
}
```

사용자 목록

```
export default function UserList({ users }: UserListProps) {
  return (
    <div style={{display:"flex"}}>
      {users.map((user, index) ⇒ (
        <UserCard
          key={index}
          name={user.name}
          role={user.role}
          skill={user.skill}
        />
      ))}
    </div>
  )
}
```

페이지 렌더링

컴포넌트 구성 후, 애플리케이션 페이지를 통해서 렌더링 한다.

```
vi src/App.tsx
import UserList from "./components/UserList"
export default function App() {
  const users = [
    {
      name: "Tang",
      role: "Infra Engineer",
      skill: "Kubernetes"
    },
```

페이지 렌더링

```
  {
    name: "Kim",
    role: "Backend Developer",
    skill: "Go / API"
  },
  {
    name: "Lee",
    role: "Frontend Developer",
    skill: "React"
  }
]
```

페이지 렌더링

```
  return (
    <div style={{padding:"20px"}}>
      <h1>User Dashboard</h1>

      <UserList users={users} />

    </div>
  )
}
```

성능 모니터링

미니 프로젝트 컴포넌트 확장

설명

앞에서 학습한 **props와 state 내용을 가지고** 아래 프로젝트 코드를 완성한다. 위의 내용을 참고해서 진행한다.

프로젝트 이름

- 시스템 메트릭 대시보드

프로젝트 목적

React에서 자주 사용하는 **props와 state 문법을 한 번에 연습한다.**

- **문자열/숫자/배열 props 전달**
- 조건부 렌더링
- map() 출력
- useState() 상태값 변경
- 버튼 이벤트 처리
- 컴포넌트 분리 및 재사용

프로젝트 생성

프로젝트 진행 전, 아래와 같이 라이브러리 및 디렉터리를 준비한다.

```
cd react-props-dashboard
mkdir -p src/components
touch src/components/MetricCard.tsx
touch src/components/MetricList.tsx
```

MetricCard.tsx

메트릭 기능을 확장하기 위해서 아래와 같이 컴포넌트를 추가한다.

```
vi src/components/MetricCard.tsx
type MetricProps = {
  title: string
  value: number
  unit: string
}
export default function MetricCard({ title, value, unit }: MetricProps) {
  let color = "green"
  if (value > 80) {
    color = "red"
  } else if (value > 50) {
    color = "orange"
  }
```

MetricCard.tsx

```
return (
  <div style={{
    border: "1px solid gray",
    padding: "15px",
    margin: "10px",
    width: "150px",
    textAlign: "center"
  }}>
```

MetricCard.tsx

```
      <h3>{title}</h3>
      <h2 style={{color}}>
        {value}{unit}
      </h2>
    </div>
  )
}
```

MetricList.tsx

메트릭 목록을 출력한다.

```
vi src/components/MetricList.tsx
import MetricCard from "./MetricCard"
type Metric = {
  title: string
  value: number
  unit: string
}
type MetricListProps = {
  metrics: Metric[]
}
export default function MetricList({ metrics }: MetricListProps) {
  return (
```

MetricList.tsx

```
    <div style={{display:"flex"}}>
      {metrics.map((m, index) => (
        <MetricCard
          key={index}
          title={m.title}
          value={m.value}
          unit={m.unit}
        />
      ))}
    </div>
  )
}
```

페이지 렌더링 통합

기존 메트릭 페이지와 다음과 같이 통합한다.

```
vi src/App.tsx
import { useState } from "react"
import UserList from "./components/UserList"
import MetricList from "./components/MetricList"

export default function App() {
  const users = [
    {
      name: "Tang",
      role: "Infra Engineer",
      skill: "Kubernetes",
    },
```

페이지 렌더링 통합

```
  {
    name: "Kim",
    role: "Backend Developer",
    skill: "Go / API",
  },
  {
    name: "Lee",
    role: "Frontend Developer",
    skill: "React",
  },
]
```

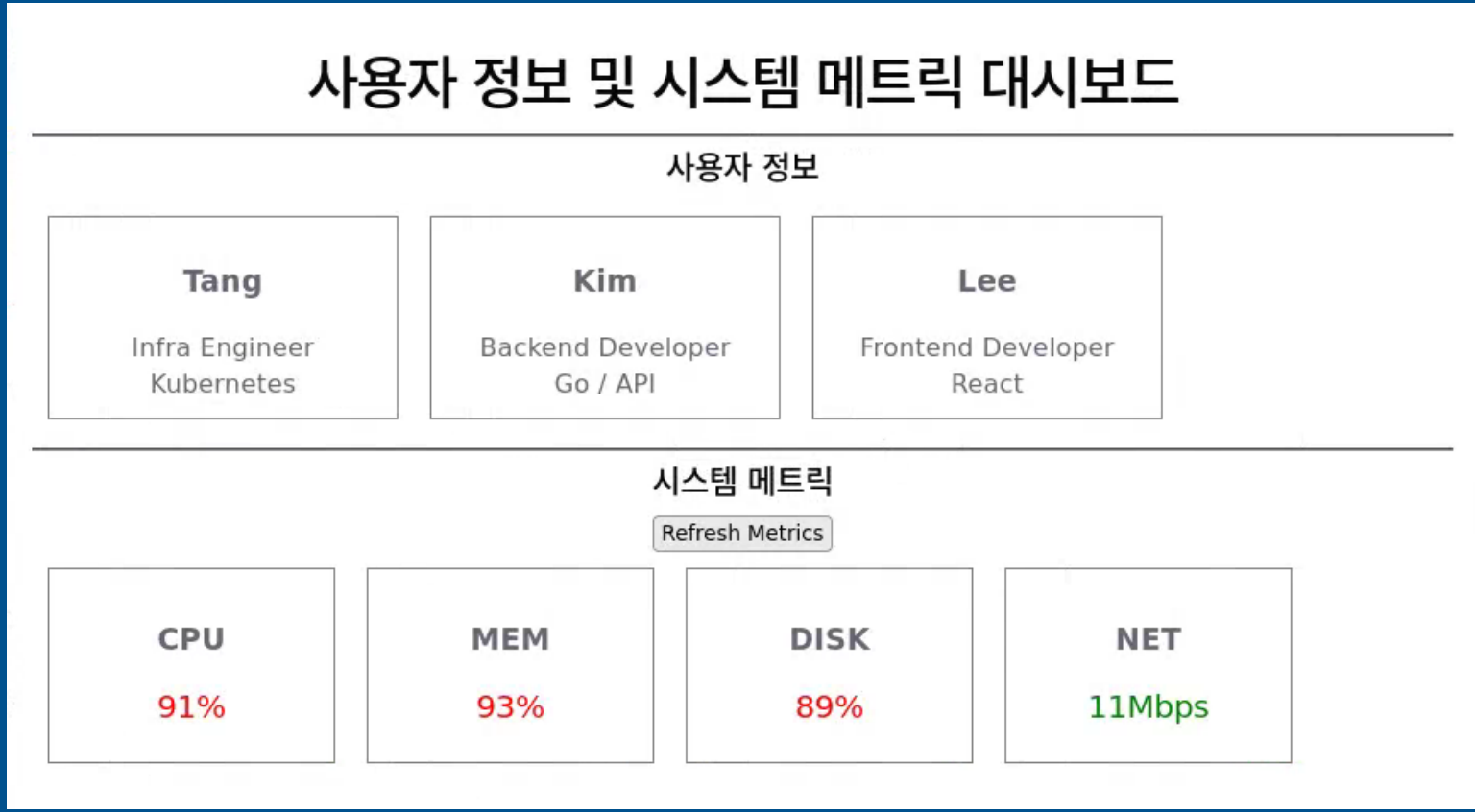
페이지 렌더링 통합

```
const [metrics, setMetrics] = useState([
  { title: "CPU", value: 30, unit: "%" },
  { title: "MEM", value: 60, unit: "%" },
  { title: "DISK", value: 80, unit: "%" },
  { title: "NET", value: 10, unit: "Mbps" },
])
const randomUpdate = () ⇒ {
  const newMetrics = metrics.map((m) ⇒ ({
     ... m,
    value: Math.floor(Math.random() * 100),
  }))

  setMetrics(newMetrics)
}
```

페이지 렌더링 통합

```
  return (
    <div style={{ padding: "20px", fontFamily: "sans-serif" }}>
      <h1>사용자 정보 및 시스템 메트릭 대시보드</h1>
      <hr />
      <h2>사용자 정보</h2>
      <UserList users={users} />
      <hr />
      <h2>시스템 메트릭</h2>
      <button onClick={randomUpdate}>Refresh Metrics</button>
      <MetricList metrics={metrics} />
    </div>
  )
}
```

결과


사용자 정보 및 시스템 메트릭 대시보드
사용자 정보
Tang
Infra Engineer
Kubernetes
Kim
Backend Developer
Go / API
Lee
Frontend Developer
React
시스템 메트릭
Refresh Metrics
CPU
91%
MEM
93%
DISK
89%
NET
11Mbps

React Event

이벤트 적용

React event를 대시보드 관점으로 설명하기

React 이벤트는 사용자의 동작을 받아서 state를 변경하고, 그 결과로 화면을 다시 갱신하는 기능. React에서는 HTML처럼 다루지만, 실제로는 JavaScript를 호출하여 동작한다.

대시보드에서는 보통 다음과 같은 방식으로 동작한다.

- Refresh 버튼 클릭
- 필터 입력
- 호스트 선택
- 자동 갱신 시작/중지
- API 다시 호출

React 이벤트 특징

1. onClick, onChange → 카멜케이스
2. 함수 형태로 전달 (문자열 X)
3. Synthetic Event 사용 (React 내부 이벤트 시스템)

최종 대시보드와 연결되는 핵심 이벤트 5개

이벤트는 자바 스크립트와 동일하다. 다만, 여기서는 최종 결과물에 사용할 이벤트만 뽑아서 사용한다. 다음과 같은 이벤트들을 사용한다.

1. onClick: 버튼 클릭 시 메트릭 갱신
2. onChange: 입력창 값 변경
3. onSubmit: 설정 저장 또는 검색 실행
4. useEffect: 페이지 처음 열릴 때 API 호출
5. setInterval: 일정 시간마다 자동 갱신

대시보드형 이벤트 예제 1: Refresh 버튼

대시보드에서 다음과 같은 이벤트를 처리한다. 아래는 예제 코드이다.

1. onClick: 마우스 클릭 이벤트
2. refreshMetrics: state 변경
3. state 변경 후, 다시, 디스플레이 렌더링

```
import { useState } from "react"
…
      <h1>Metrics Dashboard</h1>
      <button onClick={refreshMetrics}>Refresh Metrics</button>
      <ul>
        {metrics.map((m, index) ⇒ (
          <li key={index}>
            {m.title}: {m.value}
            {m.unit}
          </li>
```

시작 전…

아래와 같이 이벤트 연습 프로젝트를 생성한다.

```
npm create vite@latest react-event-lab -- --template react-ts
Framework: React
Variant: TypeScript
Install with npm and start now?: No
cd react-event-lab
npm install
mkdir -p src/examples
touch src/examples/Event01.tsx
touch src/examples/Event02.tsx
touch src/examples/Event03.tsx
touch src/examples/Event04.tsx
touch src/examples/Event05.tsx
```

시작 전…

```
touch src/examples/Event06.tsx
touch src/examples/Event07.tsx
touch src/examples/Event08.tsx
touch src/examples/Event09.tsx
touch src/examples/Event10.tsx
npm run dev -- --host 0.0.0.0 --port 8801
```

App.tsx 구성

App.tsx에 미리 컴포넌트를 작성한다. 필요 이벤트 코드 변경 시, 주석을 통해서 모듈 전환을 한다.

잊지 말자! React는 컴포넌트가 불러오지 않으면 올바르게 실행이 되지 않는다.

```
import Event01 from "./examples/Event01"
// import Event02 from "./examples/Event02"
// import Event03 from "./examples/Event03"

export default function App() {
  return <Event01 />
}
```

번거롭다면, 그냥 하나씩! :)

통합 App.tsx

코드는 깃 서버에 있는 내용을 참고한다. 총 10개의 이벤트를 구성한다.

```
import Event01 from "./examples/Event01"
import Event02 from "./examples/Event02"
import Event03 from "./examples/Event03"
import Event04 from "./examples/Event04"
import Event05 from "./examples/Event05"
import Event06 from "./examples/Event06"
import Event07 from "./examples/Event07"
import Event08 from "./examples/Event08"
import Event09 from "./examples/Event09"
import Event10 from "./examples/Event10"
```

Event01, 클릭 이벤트

React에서는 이벤트를 JSX 안에서 함수로 연결한다.

```
export default function Event01() {
  const handleClick = () => {
    alert("클릭됨!")
  }
  return (
    <div>
      <h2>Event01 – 클릭</h2>
      <button onClick={handleClick}>Click</button>
    </div>
  )
}
```

버튼 클릭 → 함수 실행 → alert 출력

Event02, 카운터

클릭 이벤트가 발생하면, 계속 숫자를 +1하여 클릭 횟수를 카운팅 한다.

```
import { useState } from "react"
export default function Event02() {
  const [count, setCount] = useState(0)
  return (
    <div>
      <h2>Event02 - 카운터</h2>
      <h3>{count}</h3>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  )
}
```

초기 값은 0으로 시작, 클릭 시 +1

클릭 → state 변경 → 화면 자동 업데이트

Event03, 입력 값 처리

입력 값을 state에 저장하고, 화면에 실시간으로 반영한다.

```
import { useState } from "react"
export default function Event03() {
  const [text, setText] = useState("")
  return (
    <div>
      <h2>Event03 - 입력</h2>
      <input onChange={(e) => setText(e.target.value)} />
      <p>{text}</p>
    </div>
  )
}
```

text: 입력값 저장소
초기값: 비어있는 문자열(useState())

e.target.value → 입력값

Event04, 조건 변경

클릭 시, CSS 스타일을 변경한다. 처음 클릭은 red, 두 번째 클릭은 black. state 값에 따라 스타일이 조건적으로 변경된다.

```
import { useState } from "react"
export default function Event04() {
  const [status, setStatus] = useState(false)
  return (
    <div>
      <h2 style={{ color: status ? "red" : "black" }}>
        상태 변경
      </h2>
      <button onClick={() => setStatus(!status)}>
        Toggle
      </button>
    </div>
  )
}
```

status: 현재상태(true/false)
초기값: false

true: red
false: black

클릭하면 true <-> false
계속 상태가 바뀜

Event05, 파라메터 전달

이벤트 발생 시, 함수를 호출하며 특정 값을 파라미터로 전달한다. 여기서는 "국현".

```
export default function Event05() {
  const sayHello = (name: string) => {
    alert(`${name}님 안녕하세요`)
  }
  return (
    <div>
      <h2>Event05 - 파라미터</h2>
      <button onClick={() => sayHello("국현")}>
        인사
      </button>
    </div>
  )
}
```

name에 값 전달 후, alter로 출력

이벤트 호출

JS처럼 호출하면 버그 발생. 반드시 => 호출

Event06, 폼 전달

onSubmit 이벤트에서 preventDefault()를 사용하여 페이지 새로고침을 방지한다.

```
export default function Event06() {
  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()
    alert("폼 제출됨")
  }
  return (
    <div>
      <h2>Event06 - Form</h2>
      <form onSubmit={handleSubmit}>
        <button type="submit">Submit</button>
      </form>
    </div>
  )
}
```

브라우저가 원래 하려고 하던 "기본 동작을 막는다"

Event07, 마우스 이벤트

마우스가 특정 영역에 들어오면 이벤트가 발생한다. **마우스 진입/이탈에 따라 상태를 변경하여 화면을 반응시키다.**

```
import { useState } from "react"

export default function Event07() {
  const [hover, setHover] = useState(false)

  return (
    <div>
      <h2>Event07 - Hover</h2>

      <div
        onMouseEnter={() => setHover(true)}
        onMouseLeave={() => setHover(false)}
```

hover = 지금 마우스 올라와 있냐?
true / false 스위치

영역 안으로 들어오면
hover = true

영역 밖으로 나가면
hover = false

Event08, 키보드 이벤트

키보드 입력 이벤트를 통해 입력된 키 값을 상태로 저장하고 화면에 반영한다. 여기서 setKey(e.key) 변수를 통해서 읽어 온 후, 화면에 출력한다.

```
import { useState } from "react"

export default function Event08() {
  const [key, setKey] = useState("")

  return (
    <div>
      <h2>Event08 - Key</h2>

      <input
        onKeyDown={(e) => setKey(e.key)}
        placeholder="키 입력"
      />
```

마지막으로 누른 키를 저장하는 변수

키 누르면

a → "a"
Enter → "Enter"
ArrowUp → "ArrowUp"

Event09, 키보드 이벤트

데이터 로딩이 완료가 되면, 화면에 완료 메시지를 출력한다. 여기서는 JavaScript함수 setTimeout()를 통해서 2000ms, 2초 대기 후 실행이 된다.

```
import { useEffect, useState } from "react"

export default function Event09() {
  const [loading, setLoading] = useState(true)
  const [msg, setMsg] = useState("")

  useEffect(() => {
    setTimeout(() => {
      setMsg("초기 메트릭 로딩 완료")
      setLoading(false)
    }, 2000)
  }, [])
```

loading = 로딩 중인지
msg = 결과 메시지

컴포넌트가 화면에 나타난 "이후" 실행됨

처음 → loading = true2초 후 → loading = false + 메시지 설정
비동기화 흉내

Event10, 자동 갱신

실제로 앞으로 만들 대시보드에서 핵심 기능. 이를 통해서 대시보드 데이터를 갱신한다. 여기서는 Math.random() 함수를 사용하여 임으로 CPU 사용률 퍼센트 생성. 2000ms, 2초에 한번씩 데이터 갱신.

```
import { useEffect, useState } from "react"

export default function Event10() {
  const [cpu, setCpu] = useState(0)

  useEffect(() => {
    const timer = setInterval(() => {
      setCpu(Math.floor(Math.random() * 100))
    }, 2000)

    return () => clearInterval(timer)
  }, [])
```

CPU 사용률 (숫자)

2초마다 실행

컴포넌트 사라질 때, 타이머 제거 하여 메모리 누수 방지

정리

React는 웹 브라우저 위에서 동작하는 라이브러리이다.

핵심은 단순하다.

이벤트 처리, 데이터 가공, API 연결. 세부 구현은 문서나 AI를 통해 언제든 보완할 수 있다.

중요한 것은 구조와 흐름을 이해하는 것이다. 인프라 엔지니어는 빠르게 이해하고 운영 서비스 적용 가능한지 불가능한지 결정하는게 중요하다.

https://react.dev/reference/react

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

인프라 엔지니어 개발 루틴
웹 브라우저 기반
↓
핵심 = 이벤트/데이터/API
↓
모르면 문서 보면 됨
↓
AI 도움 받아도 됨(퇴근은 소중!)
↓
너무 깊게 파지 마라(당신은 전문 개발자가 아니다!)

useEffect

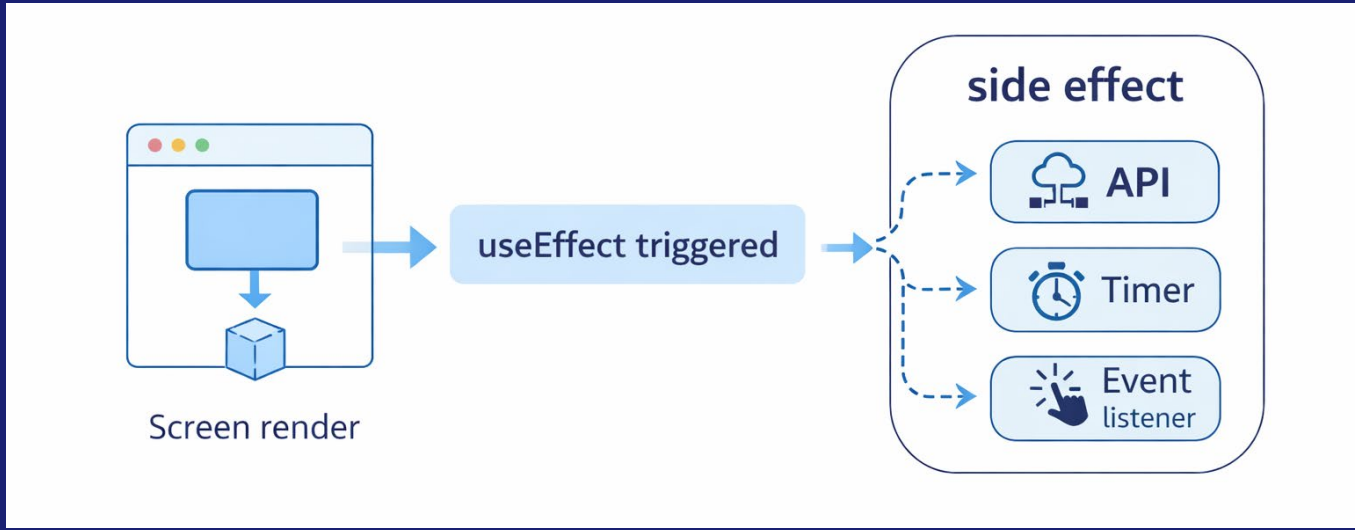
효과 적용

설명 및 동작 방식

useEffect는 무엇인가?

컴포넌트가 렌더링될 때 실행되는 함수 (사이드 이펙트 처리)

화면이 바뀔 때 실행되는 자동 이벤트. useEffect는 React가 화면(UI)을 그린 뒤,외부 작업(API, 타이머, 이벤트 등)을 실행하기 위한 Hook이다.



왜 필요한가?

React는 원래 **"UI만 그리는 역할"**만 담당한다. React는 다음과 같은 기능을 웹 브라우저를 통해서 처리한다.

1. 화면에 그림(?) 그리기
2. 상태(state) 관리

하지만, React에서 UI이외 다른 작업도 수행한다. 예를 들어서…

- API 호출
- 타이머 실행
- 로그 출력
- 이벤트 등록

이러한 이유로 useEffect를 통해서 외부 작업을 처리할 수 있도록 한다.

사용 방법

문법은 다음처럼 적용한다. **[의존성]**부분에 다음과 같이 조건이 들어간다.

1. [] → 최초 1번 실행
2. [값] → 값 변경 시
3. 실행없음 → 매 렌더링마다 실행

```
useEffect(() ⇒ {
   실행코드
}, [의존성])
```

동작방식

아래 코드를 통해서 간단하게 동작 방식을 설명한다. 동작 시 dependency(의존성) 배열을 통해서 어떠한 작업 수행할지 결정한다.

1. 컴포넌트 실행(함수)
2. return JSX(화면 그리기)
3. 화면 렌더링 완료
4. useEffect 실행

```
const [count, setCount] = useState(0)

useEffect(() => {
  console.log("useEffect 실행")
}, [])
```

useEffect에서 사이드 이펙트

사이드 이펙트는 화면 렌더링과 관련 없는 작업을 수행한다. 예를 들어서…

1. API 호출
2. console.log
3. setTimeout
4. setInterval
5. DOM 접근

이미 앞에서 우리는 몇 가지 함수와 기능을 자연스럽게 사용하였다.

적용예제

렌더링 이후에 딱 한번 실행이 필요하는 경우, 아래와 같이 구성한다. 아래 코드 조건은 다음과 같다.

- 2초 후에 "1번만 실행"라는 메시지 출력

```
useEffect(() => {
  setTimeout(() => {
    console.log("1번만 실행")
  }, 2000)
}, [])
```

```
// 아래 코드는 위와 비슷하지만, 페이지가 렌더링 될때마다 계속 실행이 된다.
setTimeout(() => {
  console.log("실행")
}, 2000)
```

프로젝트 준비

다음과 같이 프로젝트를 준비한다.

```
npm create vite@latest react-useeffect-lab -- --template react-ts
Framework: React
Variant: TypeScript
Install with npm and start now?: No
cd react-useeffect-lab
npm install
mkdir -p src/examples
touch src/examples/Example{01..10}.tsx
```

App.tsx

App.tsx에 미리 컴포넌트를 작성한다. 필요 이벤트 코드 변경 시, 주석을 통해서 모듈 전환을 한다. 잊지 말자!
React는 컴포넌트가 불러오지 않으면 올바르게 실행이 되지 않는다.

```
import { useState } from "react"

import Example01 from "./examples/Example01"
import Example02 from "./examples/Example02"
import Example03 from "./examples/Example03"
import Example04 from "./examples/Example04"
import Example05 from "./examples/Example05"
import Example06 from "./examples/Example06"
import Example07 from "./examples/Example07"
import Example08 from "./examples/Example08"
import Example09 from "./examples/Example09"
import Example10 from "./examples/Example10"
```

Example 1: 최초 실행 (기본)

기본적인 useEffect 예제. 모듈을 불러온 후, 아래와 같이 적용한다.

```
import { useEffect } from "react"

export default function Example01() {
  useEffect(() => {
    console.log("처음 실행됨")
  }, [])

  return <h2>Example01</h2>
}
```

로그 찍기
API 호출
타이머
DOM 조작

이 함수는 **컴포넌트가 화면에 처음 등장할 때 실행**

Example 2: 상태 변경 감지

이번에는 상태를 변경하는 이벤트. 모듈 두 개를 불러온 후, 적용한다.

```
import { useEffect, useState } from "react"
export default function Example02() {
  const [count, setCount] = useState(0)
  useEffect(() => {
    console.log("count 변경:", count)
  }, [count])
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  )
}
```

count: 현재 값
setCount: 값을 바꾸는 스위치

count가 바뀔 때마다 실행됨

버튼 클릭 → count 증가 → React 재렌더링 → useEffect 실행

Example 3: 매 렌더링 실행

이번에는 렌더링이 매 클릭 시 발생한다. 의존성 배열이 없기 때문에 이벤트가 발생 시, 매번 렌더링이 발생한다.

```
import { useEffect, useState } from "react"
export default function Example03() {
  const [count, setCount] = useState(0)
  useEffect(() ⇒ {
    console.log("렌더링 발생")
  })
  return (
    <button onClick={() ⇒ setCount(count + 1)}>
      {count}
    </button>
  )
}
```

```
useEffect(() => {
  setCount(count + 1)
})
```

렌더링 → setCount → 렌더링 → setCount → 무한루프

```
useEffect(() => {
  console.log("렌더링 발생")
}, [])
```

정상 코드

DUSTBOX

Example 4: 타이머 (setTimeout)

2초 후에 이벤트 발생. 여기서는 useEffect만 사용하기 때문에 타이머 기능만 동작한다.

```
import { useEffect } from "react"

export default function Example04() {
  useEffect(() => {
    setTimeout(() => {
      console.log("2초 후 실행")
    }, 2000)
  }, [])

  return <h2>Timer</h2>
}
```

⚠️ 그런데 중요한 문제 하나
👉 컴포넌트가 2초 안에 사라지면?
타이머는 계속 살아있음 😱
메모리 낭비/예상 못한 실행 발생

```
useEffect(() => {
  const timer = setTimeout(() => {
    console.log("2초 후 실행")
  }, 2000)
  return () => {
    clearTimeout(timer)
  }
}, [])
```

DUSTBOX

Example 5: 자동 갱신 (setInterval)

매 초마다 시간을 업데이트 한다.

```
import { useEffect, useState } from "react"

export default function Example05() {
  const [time, setTime] = useState(0)
  useEffect(() => {
    const timer = setInterval(() => {
      setTime(t => t + 1)
    }, 1000)
    return () => clearInterval(timer)
  }, [])
  return <h2>{time}</h2>
}
```

권장: `setTime(t => t + 1)`

1000ms=1초

타이머 제거

Example 6: API 응답 시뮬레이션

실제 API는 아직 연결하지 않았고, API 응답이 온 것처럼 가정하여 데이터를 불러오는 흐름만 간단히 구성했다.

```react_example_06.jsx
import { useEffect, useState } from "react"
export default function Example06() {
  const [data, setData] = useState("Loading ... ")
  useEffect(() => {
    setTimeout(() => {
      setData("데이터 도착")
    }, 2000)
    return () => clearTimeout(timer)
  }, [])
  return <h2>{data}</h2>
}
```

컴포넌트 실행
↓
data = "Loading ... "
↓
화면에 Loading ... 출력
↓
useEffect 실행
↓
2초 대기
↓
setData("데이터 도착")
↓
재렌더링
↓
화면에 "데이터 도착" 출력

Example 7: loading 상태 추가

Example 6번에서 확장. **데이터 로딩**이 완료가 되면, **완료**라는 메시지를 출력.

```
import { useEffect, useState } from "react"
export default function Example07() {
  const [loading, setLoading] = useState(true)
  useEffect(() => {
    setTimeout(() => {
      setLoading(false)
    }, 2000)
  }, [])
  return (
    <div>
      {loading ? <p>Loading ... </p> : <p>완료</p>}
    </div>
  )
```

2000ms=2초

[컴포넌트 시작]
↓
loading = true
↓
화면: "Loading ... "
↓
2초 후
↓
setLoading(false)
↓
렌더링 발생
↓
화면: "완료"

Example 8: 조건부 실행

조건에 따라서 실행이 된다. 여기서는 count 변수가 5보다 큰 경우 실행이 된다.

```
import { useEffect, useState } from "react"
export default function Example08() {
  const [count, setCount] = useState(0)
  useEffect(() => {
    if (count > 5) {
      console.log("5 이상!")
    }
  }, [count])
  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  )
```

의존성: 실행 시, count 변수가 있어야 한다.

Example 9: cleanup

useEffect에서 실행 혹은 사용한 작업을 정리(혹은 제거)하는 함수이다. 아래와 같이 모든 작업을 제거가 가능하다.

```
import { useEffect } from "react"

export default function Example09() {
  useEffect(() => {
    console.log("실행")
    return () => {
      console.log("정리(cleanup)")
    }
  }, [])

  return <h2>Cleanup Test</h2>
}
```

화면에 들어옴 -> 실행

완료된 작업을 메모리에서 초기화 한다.

Example 9: cleanup+1

cleanup를 적용한 코드는 보통 다음과 같이 동작한다.

```
import { useEffect, useState } from "react"

export default function App() {
  const [count, setCount] = useState(0)

  useEffect(() => {
    const timer = setInterval(() => {
      console.log("실행 중")
    }, 1000)

    return () => {
      clearInterval(timer)
    }
```

1초 후 "실행 중" 메시지 출력

실행 후 Interval() 값 삭제. 이 부분이 없으면 메모리 누수 발생

Example 9: cleanup+2

다음처럼 코드가 구성이 되면, 웹 브라우저에 이벤트 리스너가 누적으로 적용 및 실행이 되기 때문에 점점 브라우저가 느려진다.

```
useEffect(() => {
  const handleResize = () => {
    console.log("화면 변경")
  }

  window.addEventListener("resize", handleResize)

  return () => {
    window.removeEventListener("resize", handleResize)
  }
}, [])
```

이 부분이 없으면 이벤트 리스너가 계속 브라우저 이벤트가 계속 생성이 된다.

DUSTBOX

Example 10: Metrics Dashboard 연결

현재 대시보드 데이터가 없기 때문에 임시로 Math.random()를 사용하여 백분율 기준으로 임의의 데이터를 생성한다.

```
import { useEffect, useState } from "react"

export default function Example10() {
  const [cpu, setCpu] = useState(0)

  useEffect(() => {
    const timer = setInterval(() => {
      setCpu(Math.floor(Math.random() * 100))
    }, 2000)
    return () => clearInterval(timer)
  }, [])
```

현재는 API를 통해서 데이터를 가져오지 않기 때문에 임시로 구성한 데이터를 출력한다.

React에서 API 다루기

GO언어 기반 NODE 모니터링 시스템 작성

2026-03-19

왜 GO언어를 사용하였는가?

인프라엔지니어의 개발

왜 Go를 쓰는가 (핵심만)

Go는 **"내가(AKA 강사가) 좋아서"** 쓰는 언어가 아니다.

👉 **운영 환경에서 비용 대비 효율이 가장 안정적으로 나오기 때문에 선택된다.**

판단 기준 4개로 보자.

1. 동시성

인프라 시스템은 동시성이 정말로 중요하다. 기존에 사용하던 C언어 경우에는 동시성을 구현하기 위해서 너무 많은 시간이 필요하며, 거기에 디버깅 부분까지 고려해야 한다. 하지만, GO언어는 기본적으로 동시성 기반으로 만들어진 언어라서, 다음과 같은 부분에 대해서 장점이 있다.

- 수천 개의 작업을 가볍게 처리 가능(대표적인 케이스 gRPC)
- 네트워크/API/에이전트/컨트롤러에 최적

왜 Go를 쓰는가 (핵심만)

좀 더 쉽게 표로 정리하면 다음과 같다.

인프라 엔지니어는 개발자와 **"빠르게 안정적으로 만들 수 있느냐?"**가 언어 사용 시 관건이다.

언어	동시성 난이도
C/C++	매우 어려움 (thread, mutex)
Python	GIL(Gloabal Interpreter Lock) 제약
Rust	강력하지만 학습 난이도 높음
Go	가장 단순하면서 충분히 강력

배포

소프트웨어 엔지니어와 다르게, 인프라 엔지니어는 **빠른배포**에 매우 민감하다. 왜냐면, 다루는 시스템이 한 대가 아니라 최소 수십대에서 최대 수천대가 될 수가 있다.

이러한 이유로 C/C++처럼 **glibc를 통한 라이브러리 링크는 매우 비효율적**이다. 라이브러리 링크의 최대 장점은 자원의 재활용 이지만, 버전에 따라서 **헤더(header)**가 다르기 때문에 올바르게 동작이 되지 않거나 메모리 오류가 자주 발생한다.

하지만, GO언어는 기본적으로 **단일 바이너리로 배포**가 가능하기 때문에 의존성이 C/C++비해서 라이브러리 의존성이 매우 낮으면 **컨테이너/가상/서버 형태**로 어디든지 바로 실행이 가능하다.

언어	배포
Python	runtime 필요
Node	npm dependency
C/C++	컴파일/라이브러리 지옥
Go	그냥 복사해서 실행

표준 라이브러리

별도의 추가 라이브러리가 없이 대다수 기능은 기본적으로 GO 언어에서 빌트인(built-in)형태로 제공한다. 예를 들어서 대표적인 라이브러리는 다음과 같다.

- http
- json
- net
- file

이러한 이유로 Python 및 NodeJS에 비해서 매우 간단하다.

왜 C/C++ 대신 쓰는가

꽤나 차갑고 냉정하게 말한다. 본인은 C언어 기반으로 커널 패치 및 모듈 개발을 하였다. 하지만, 이 기반으로 인프라 운영을 위한 애플리케이션을 작성한다?

이는 매우 형편없고 비효율적인 선택이다. C/C++은 기본적으로 메모리 관리 및 디버그라는게 항상 따라다니는 매우 고-난이도의 언어이다. 또한, 시스템별로 각기 다른 라이브러리 및 컴파일러를 사용하기 때문에, 이에 따라서 코드 호환성도 많이 달라진다.

C/C++은 다음과 같은 경우에는 사용을 권장한다.

- OS
- 커널/드라이버
- 초고성능 시스템

일반적인 API/클라우드/에이전트 시스템에서는 C/C++를 과하다. 또한, 현재 최근 언어들은 C언어 못지 않는 **기계어(compilation)** 성능을 내고 있기 때문에, 점점 C/C++ 사용은 이전에 비해서 줄어드는 상황이다.

GO/PYTHON/RUST

다른 언어와 비교하면 다음과 같다. 여기부터 **외부 논문 및 개인적인 생각이 포함**이 된다. 참고만 하자 :)

파이썬

파이썬은 높은 생산성과 학습 용이성을 갖춘 언어로, 데이터 처리 및 자동화 영역에서 널리 활용된다.

- GIL(Global Interpreter Lock)로 인해 멀티코어 활용에 제약이 있음
- 인터프리터 기반으로 인해 실행 성능에 한계가 있음
- 런타임 및 의존성 관리(venv, pip)가 필요하여 배포 환경 구성이 복잡할 수 있음

따라서 단일 도메인, 데이터 처리, 자동화 작업에는 매우 적합하지만, 고성능이 요구되는 대규모 시스템이나 고동시성 서비스에서는 구조적 제약이 존재한다.

GO/PYTHON/RUST

RUST

Rust는 메모리 안정성과 높은 성능을 동시에 제공하는 시스템 프로그래밍 언어이다.

특히 **소유권(Ownership) 모델**을 기반으로 **런타임 오버헤드 없이 메모리 안정성을 확보**할 수 있으며, **동시성 환경에서도 데이터 경쟁(Data Race)을 컴파일 단계에서 방지**할 수 있다.

그러나 다음과 같은 한계가 존재한다.

- **소유권 및 생명주기(Lifetime) 개념으로 인해 학습 난이도가 높음**
- **컴파일 에러 메시지가 복잡하여 초기 생산성이 낮을 수 있음**
- **빠른 프로토타이핑 및 개발 속도 측면에서 부담이 존재함**

따라서 고성능이 요구되거나 메모리 안정성이 중요한 시스템 영역에서는 매우 적합하지만, 일반적인 서비스 개발이나 빠른 개발이 요구되는 환경에서는 진입 장벽이 존재한다.

GO/PYTHON/RUST

GO

Go는 단순한 문법과 강력한 동시성 모델을 기반으로, 네트워크 서비스 및 인프라 환경에 적합하게 설계된 언어이다. **goroutine과 channel**을 통해 **경량 스레드 기반의 동시성 처리를 지원**하며, 단일 바이너리 형태로 빌드되어 배포가 용이하다.

또한 표준 라이브러리에서 네트워크, HTTP, JSON 처리 기능을 기본적으로 제공하여 추가적인 의존성 없이 서비스 구현이 가능하다.

그러나 다음과 같은 한계가 존재한다.

- 제네릭 및 타입 시스템이 상대적으로 단순하여 복잡한 추상화에는 한계가 있음
- 메모리 제어가 제한적이며, 저수준 시스템 제어에는 부적합함
- 언어 기능이 제한적이기 때문에 표현력이 부족하다고 느껴질 수 있음

따라서 Go는 고성능 분산 시스템, API 서버, 클라우드 인프라 환경에 매우 적합하며, 개발 생산성과 운영 안정성을 동시에 요구하는 환경에서 효과적인 선택이 될 수 있다.

정리

1. Python은 생산성과 유연성
2. Rust는 안정성과 성능
3. Go는 생산성과 운영 효율의 균형에 초점을 둔 언어

API 호출 테스트 준비

소개

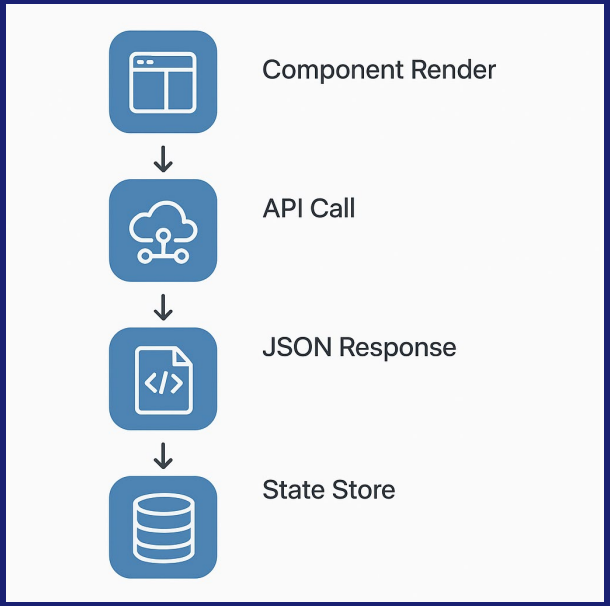
컴파일 및 테스트

호출 테스트(React)

왜 API를 사용하는가?

자, 간단하게 React 기반으로 UI구성을 간단히 다루었다. 이제 여기에 API 데이터 처리 부분만 추가가 되면 모든 내용은 완료가 된다.

대다수 인프라 관리 소프트웨어는 API 기반으로 데이터를 전달 및 처리하기 때문에, 대시보드에 출력하기 위해서 API 데이터 핸들링을 여기에서 다룬다. 보통 다음과 같은 흐름으로 데이터 처리가 된다.



왜 필요한가?

기존에 인프라 엔지니어 입장에서 웹 페이지 기반으로 대시보드 작성 시, 다음과 같은 단계를 진행하였다.

1. 데이터베이스 설계
2. 각 프로그램에서 사용하는 SQL 레코드 불러오기 및 전환
3. DBC(Database Connector)를 통해서 사용하는 WEB Application Language에 데이터 가공
4. 가공된 데이터를 배열 혹은 반복문을 통해서 HTML으로 출력
5. 실시간 출력이 필요한 경우 Native JavaScript를 통해서 AJAX나 혹은 DOM 다루기

이러한 단계를 거치면서 작성하였기 때문에, 운영시간보다 개발 시간이 더 많이 소모가 되었다. 현재 사용하는 대다수 **인프라 소프트웨어는 API(Application Programmable Interface)를 통해서 데이터 전달**이 가능하기 때문에 더 빠른 인프라 소프트웨어 개발이 가능하다.

왜 필요한가?

앞에서 만든 예제 및 간단한 대시보드 미니프로젝트에서는 가짜 데이터(Math.random())를 사용하여, 마치 CPU나 MEM가 실제로 사용하는 것처럼 출력하였다.

하지만, 이제는 실제 시스템 데이터(혹은 서버 데이터)를 통해서 다음과 같은 사용율을 가져온다.

- CPU 사용률
- 메모리 사용률
- 디스크 사용률
- 네트워크 트래픽

위의 정보를 JSON 형태로 API를 통해서 전달 받아서, 이걸 React로 다시 렌더링해서 출력한다.

React에서 API 처리할 때 핵심 4개

이 교육에서는 4가지 핵심만 잡고 진행한다.

1. useEffect: 초기 화면 렌더링 후 API 호출
2. fetch(아직 다루지 않음): HTTP 요청 수행
3. useState: 받아온 데이터 저장
4. loading/error: 사용자에게 현재 상태 표시

데이터 가져오기

지금까지 UI부분에 대해서 이야기 하였고, API를 통해서 처리할 데이터를 가져오는 방법에 대해서는 아직 진행하지 않았다. 이 슬라이드에서 fetch함수를 통해서 API요청 및 데이터를 가져오는 방법에 대해서 이야기 해보도록 한다.

동작 흐름은 다음과 같다.

React ➡️ fetch ➡️ API 서버 ➡️ JSON 응답 ➡️ React

fetch모듈을 사용하여 구성 시, 다음과 같이 코드를 작성한다.

```
fetch("URL")
  .then((response) ⇒ response.json())
  .then((data) ⇒ {
    console.log(data)
  })
  .catch((error) ⇒ {
    console.error(error)
  })
```

동작흐름

1. fetch 요청
2. 서버 응답 (Response 객체)
3. JSON 변환
4. 데이터 사용

위와 같은 순서로 동작한다. 여기서 중요한 부분은

response.json()은 비동기로 동작해야 된다. 이러한 이유로 .then()라는 함수를 사용한다.

❗ 여기서 사용하는 함수는 전부 자바스크립트 함수 입니다. ❗

간단하게 백엔드 구성

아직, 우리는 백엔드 구성을 하지 않았다. 여기서 간단하게 GO언어 기반으로 백엔드 구성한다. 디렉터리 구조는 아래와 같이 구성한다.

```
mkdir -p backend/handlers
touch backend/main.go
touch backend/handlers/{hello.go, metrics.go}
git clone https://git.dustbox.kr/tang/demo-go-metrics-api.git
tree -L2
backend/
 ├── main.go
 ├── handlers/
 │    ├── hello.go
 │    └── metrics.go
```

hello.go

간단하게 API호출에 응답 할 "Hello from Go Backend"라는 컨텍스트 핸들러를 생성한다.

```
vi hello.go
package handlers
import (
        "net/http"
        "github.com/gin-gonic/gin"
)
func HelloHandler(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
                "message": "Hello from Go Backend 🚀",
        })
}
```

main.go

컨텍스트 호출 할 URL를 GET으로 처리한다. 메인함수(main())는 다음과 같이 동작한다.

```
vi main.go
func main() {
        r := gin.Default()
        r.Use(cors.Default())
        r.GET("/api/health", handlers.HealthHandler)
        r.GET("/api/hello", handlers.HelloHandler)
        r.GET("/api/metrics/summary", handlers.SummaryHandler)
        r.Run(":8080")
}
```

health.go

외부에서 /health 컨텍스트로 접근하면 아래와 같이 메시지를 출력한다.

```
vi health.go
package handlers
import (
        "net/http"
        "github.com/gin-gonic/gin"
)
func HealthHandler(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
                "status": "ok",
        })
}
```

summary.go

다음과 같은 자원 정보를 사용자에게 전달한다.

```
Hostname:      hostInfo.Hostname,
UptimeSeconds: hostInfo.Uptime,
CPUPercent:    cpuPercent,
MemTotalMB:    memInfo.Total / 1024 / 1024,
MemUsedMB:     memInfo.Used / 1024 / 1024,
MemPercent:    memInfo.UsedPercent,
DiskTotalGB:   diskInfo.Total / 1024 / 1024 / 1024,
DiskUsedGB:    diskInfo.Used / 1024 / 1024 / 1024,
DiskPercent:   diskInfo.UsedPercent,
NetRxBytes:    netRx,
NetTxBytes:    netTx,
CollectedAt:   time.Now().Format(time.RFC3339),
```

백엔드 컴파일 및 실행

백인드 컴파일 및 실행은 다음처럼 진행한다.

```
dnf install git golang -y
git clone https://git.dustbox.kr/tang/demo-go-metrics-api.git
go mod tidy
go build
```

백엔드 확인

아래와 같이 프로그램을 실행한다. 잘 동작이 되면, 백엔드에 대해서 간단하게 설명 후, React와 연결 및 테스트를 진행한다.

```
curl localhost:8080/health
curl localhost:8080/api/hello
curl localhost:8080/api/metrics/summary
```

React에서 API 핸들링

React에서 API 다루기

2026-03-19

API 데이터 핸들링 시작

우리는 API를 통해서 다음과 같이 화면구성을 할 계획이다. 대시보드 다음과 같은 내용들이 출력이 된다.

1. Hostname
2. Uptime
3. CPU
4. MEM
5. DISK
6. NET RX/TX
7. Refresh 버튼

사용하는 라이브러리

앞서 학습한 자바 및 React 라이브러리를 사용하여 애플리케이션을 구성한다. 다만, 이전과 차이라고 하면 fetch 라이브러리를 사용하여 API를 가져온다.

코드가 매우 길게 구성이 되어 있기 때문에, 여기서 부터는 필요한 코드 부분은 PPT에 노출한다.

- React → UI 구성
- TypeScript → 타입 안정성 확보
- Vite → 개발 서버 및 빌드 도구
- fetch (기본 API) → 백엔드 데이터 호출

React 디렉터리 준비

사용할 프론트엔드 프레임워크 디렉터리를 생성한다.

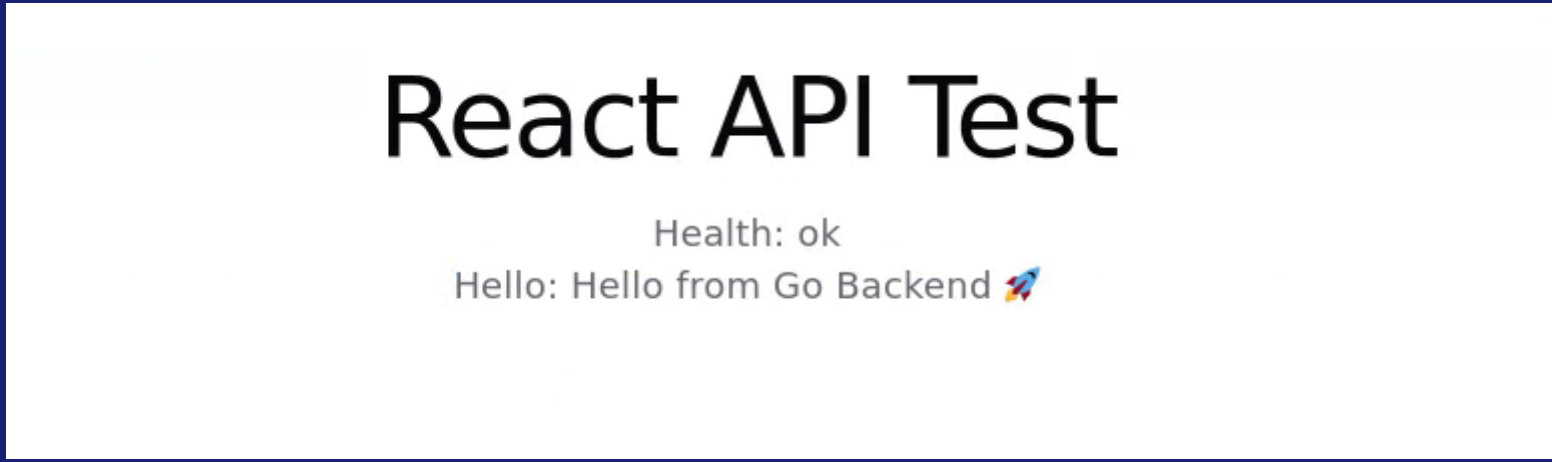
```
npm create vite@latest react-api-test -- --template react-ts
cd react-api-test
npm install
npm run dev -- --host 0.0.0.0 --port 5173

cd react-api-test
mkdir -p src/components
```

React 디렉터리 준비

```
import { useEffect, useState } from "react"
export default function App() {
  const [health, setHealth] = useState("")
  const [hello, setHello] = useState("")
  useEffect(() => {
    fetch("http://<IP_ADDRESS>:8080/api/health")
      .then((res) => res.json())
      .then((data) => setHealth(data.status))

    fetch("http://<IP_ADDRESS>:8080/api/hello")
      .then((res) => res.json())
      .then((data) => setHello(data.message))
  }, [])
```

예상 결과

문제가 없으면 React에서 fetch()함수를 통해서 데이터를 불러와서 다음과 같이 출력한다.



좀 더 강화하기

백엔드와 프론트 엔드가 문제없이 동작이 된다. 이제는 백엔드에서 API 데이터를 가져오지 못하면 다음과 같은 메시지가 출력 되도록 구성한다

1. health API 호출 실패
2. hello API 호출 실패

기존 코드는 그대로 사용하고 .then를 통해서 res!=200이 아니면 "health/hello API 호출 실패"라는 메시지를 출력한다.

예외처리 추가 및 강화

```
fetch("http://10.0.4.142:8080/api/health")
  .then((res) => {
    if (!res.ok) {
      throw new Error("health API 호출 실패")
    }
    return res.json()
  })
  .catch((err) => setError((prev) => prev + `Health Error: ${err.message} `))
fetch("http://10.0.4.142:8080/api/hello")
  .then((res) => {
    if (!res.ok) {
      throw new Error("hello API 호출 실패")
    }
    return res.json()
  })
  .catch((err) => setError((prev) => prev + `Hello Error: ${err.message}`))
```

1차 완성!

자, 이제는 fetch()를 통해서 2개의 데이터를 문제없이 가져왔다. 이번에는 메트릭 정보를 가져와야 한다. 메트릭 정보는 다음과 같은 조건을 기반으로 구성이 된다.

1. 매 2초마다 최신 정보를 갱신한다
2. 문제가 있으면 "데이터 가져오기 실패"라는 메시지를 출력

메트릭 정보 가져오기

이제, 본 목적인 메트릭 정보를 가져온다. 올바르게 백엔드가 동작하는지 확인하기 위해서 아래 명령어로 확인한다. 다음과 같이 JSON으로 값이 출력이 되면, 백-엔드는 올바르게 동작하고 있다.

```
curl http://localhost:8080/api/metrics/summary
{"hostname":"metric-
node.novalocal","uptimeSeconds":10382,"cpuPercent":1.0000000000218279,"memT
otalMB":956,"memUsedMB":195,"memPercent":20.406530285648966,"diskTotalGB":4
,"diskUsedGB":2,"diskPercent":50.17260073716883,"netRxBytes":411884705,"net
TxBytes":6355003,"collectedAt":"2026-03-18T10:50:17Z"}
```

코드 강화

메트릭 정보를 가져오기 위해서 다음과 같이 변수를 선언한다.

```
type SummaryResponse = {
  hostname: string
  uptimeSeconds: number
  cpuPercent: number
  memTotalMB: number
  memUsedMB: number
  memPercent: number
  diskTotalGB: number
  diskUsedGB: number
  diskPercent: number
  netRxBytes: number
  netTxBytes: number
  collectedAt: string
}
```

DUSTBOX

코드 강화

주소가 자주 변경이 되기 때문에 상수로 고정한다. 이외 나머지 값에 대해서는 useState()를 통해서 React에서 사용이 가능하도록 한다.

summary 경우에는 빈 값이 아닌 경우, 참으로 처리한다. setLoading()는 모든 데이터가 불러오기 전까지 화면을 그리지 않는다.

```
const [health, setHealth] = useState("")
const [hello, setHello] = useState("")
const [summary, setSummary] = useState<SummaryResponse | null>(null)

const [loading, setLoading] = useState(false)
const [error, setError] = useState("")

const BASE_URL = "http://10.0.4.142:8080"
```

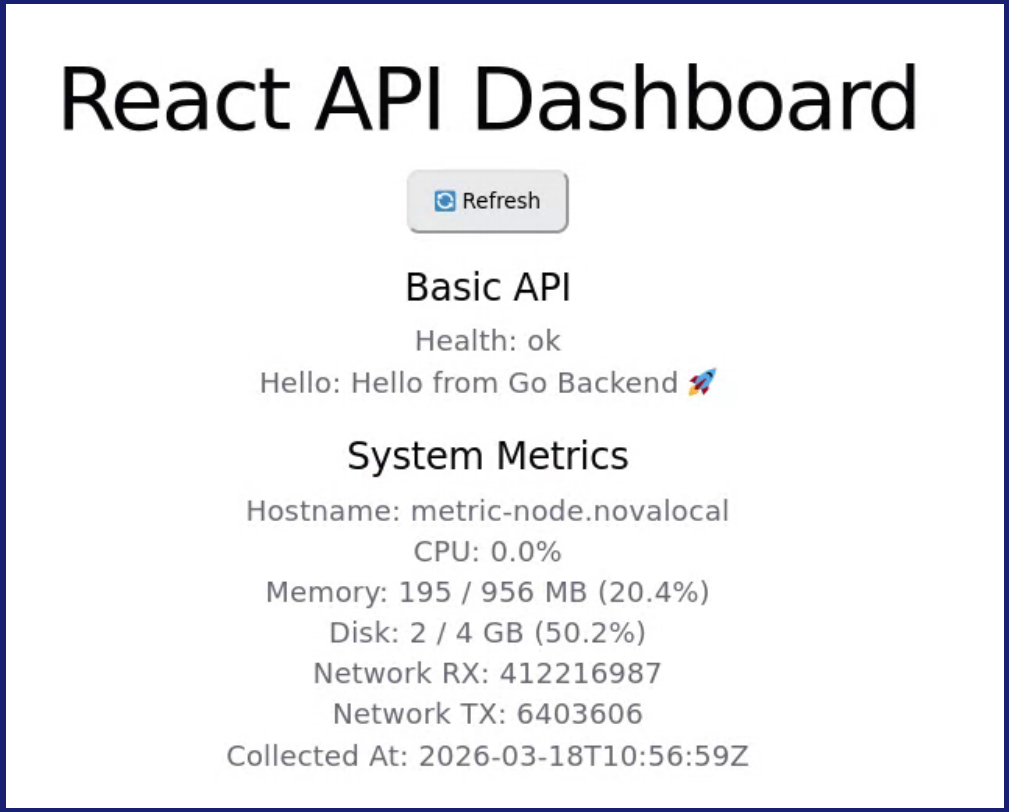
코드 강화

나머지 부분은 이전과 거의 동일하다. 다만, 이전에 API 호출 시 매번 URL를 호출하였지만 이걸 변수로 변경하고, 메트릭 URL를 추가한다.

```
// metrics
const summaryRes = await fetch(`${BASE_URL}/api/metrics/summary`)
if (!summaryRes.ok) throw new Error("metrics API 실패")
const summaryData: SummaryResponse = await summaryRes.json()
setSummary(summaryData)
```

예상 결과

문제가 없으면 다음처럼 화면이 렌더링이 되면서 결과가 출력이 된다.



마지막 프로젝트

LLM CHAT REACT with llama.cpp

목적 설명

마지막 프로젝트는 CPU 기반으로 추론이 가능한 llama.cpp 서비스를 React로 구성한다. 간단하게 인스턴스 구성 후, 다음과 같은 인프라 서비스를 구성한다.

1. llama.cpp를 Pod기반으로 서비스 구성
2. OpenAI API v1를 활용하여 React에서 대화 콘솔 제공
3. 콘솔을 통해서 시스템 명령어 및 장애 처리 지원

강사와 함께 간단하게 llama.cpp를 CPU기반으로 구성 및 모델 셋팅을 같이 진행한다.

준비

이전과 동일하게 프로젝트를 생성한다.

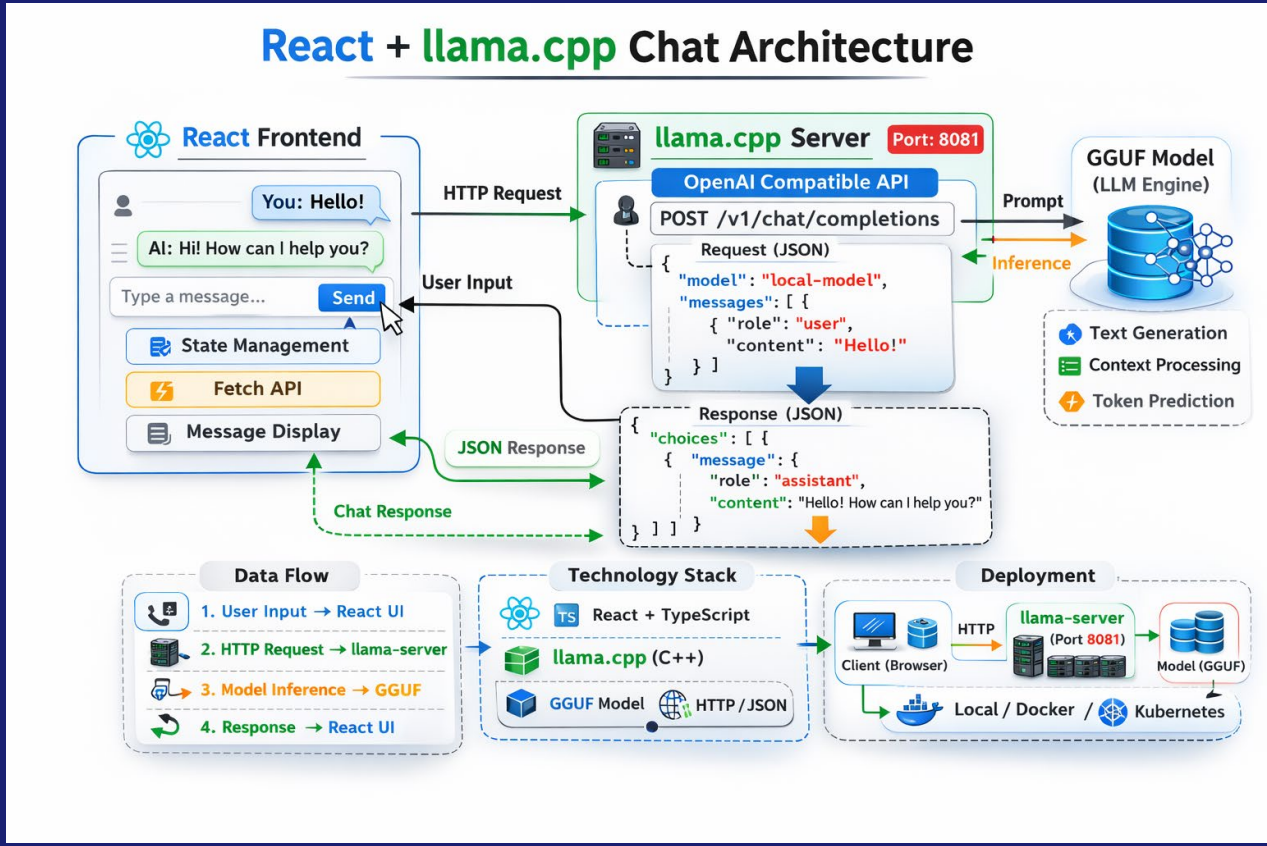
```
npm create vite@latest llama-chat -- --template react-ts
cd llama-chat
npm install
npm run dev -- --host 0.0.0.0 --port 5173
```

준비

디렉터리 구조는 다음과 같이 구성이 된다.

```
llama-chat/
├── index.html
├── package.json
├── vite.config.ts
└── src/
    ├── App.tsx    ← 여기에 코드 붙임
    └── main.tsx
```

채팅 서비스 아키텍처



2026-03-19

소스코드 설명

소스코드가 매우 길어서 중요 부분만 설명.

```
try {
      const res = await fetch(`${BASE_URL}/chat/completions`, {
…
        body: JSON.stringify({
          model: "local-model",
          messages: nextMessages,
          temperature: 0.7,
          max_tokens: 256,
        }),
      })
```

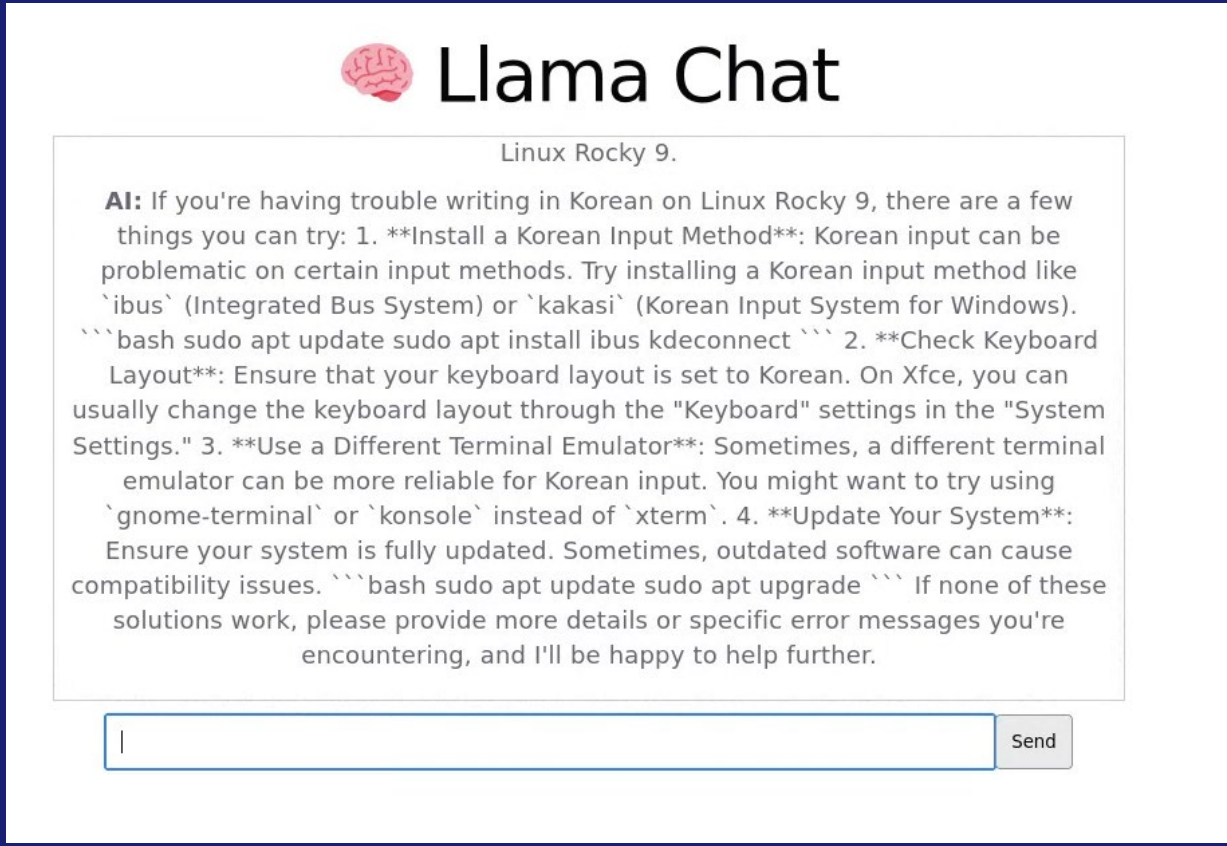
소스코드 설명

```
setMessages((prev) ⇒ [
        ... prev,
        { role: "assistant", content: answer },
      ])
    } catch (e) {
      setMessages((prev) ⇒ [
        ... prev,
        { role: "assistant", content: "에러 발생" },
      ])
    } finally {
      setLoading(false)
    }
  }
```

소스코드 설명

```
<input
  value={input}
  onChange={(e) => setInput(e.target.value)}
  onKeyDown={(e) => e.key === "Enter" && sendMessage()}
  style={{ width: "80%", padding: "10px" }}
/>
```

결과


Llama Chat
Linux Rocky 9.
AI: If you're having trouble writing in Korean on Linux Rocky 9, there are a few things you can try: 1. **Install a Korean Input Method**: Korean input can be problematic on certain input methods. Try installing a Korean input method like `ibus` (Integrated Bus System) or `kakasi` (Korean Input System for Windows). ```bash sudo apt update sudo apt install ibus kdeconnect ``` 2. **Check Keyboard Layout**: Ensure that your keyboard layout is set to Korean. On Xfce, you can usually change the keyboard layout through the "Keyboard" settings in the "System Settings." 3. **Use a Different Terminal Emulator**: Sometimes, a different terminal emulator can be more reliable for Korean input. You might want to try using `gnome-terminal` or `konsole` instead of `xterm`. 4. **Update Your System**: Ensure your system is fully updated. Sometimes, outdated software can cause compatibility issues. ```bash sudo apt update sudo apt upgrade ``` If none of these solutions work, please provide more details or specific error messages you're encountering, and I'll be happy to help further.
Send

Podman 컨테이너 패키징

애플리케이션을 서비스로 구성

컨테이너(Container) vs 가상머신(VM)

컨테이너는 애플리케이션과 실행에 필요한 라이브러리 및 환경을 하나의 패키지로 묶어, 운영체제 커널을 공유하면서 빠르고 가볍게 실행되는 기술이다.

반면 **가상머신은 하이퍼바이저 위에서 각각 독립적인 운영체제**를 포함하여 실행되므로, 더 높은 격리성과 안정성을 제공하지만 상대적으로 무겁고 자원 사용량이 많다.

컨테이너는 OS를 포함하지 않기 때문에 실행 속도가 매우 빠르고, 동일한 환경을 어디서든 재현할 수 있어 DevOps 및 CI/CD 환경에 적합하다. 반대로 가상머신은 완전한 OS 환경을 제공하기 때문에 서로 다른 OS를 동시에 운영하거나, 강한 보안 격리가 필요한 환경에 적합하다.

결론적으로, 컨테이너는 **"빠르고 가벼운 애플리케이션 실행 환경"**에 최적화되어 있으며, 가상머신은 **"완전한 독립 시스템"**을 구성하는 데 적합한 기술이다.

Podman vs Docker

Docker는 컨테이너 실행을 위해 중앙 데몬(dockerd)을 사용하는 구조로, 클라이언트가 데몬을 통해 컨테이너를 제어하는 방식이다. 이 구조는 사용이 간편하고 생태계가 풍부하지만, 데몬이 단일 장애 지점(SPOF)이 될 수 있으며 보안 측면에서 root 권한 의존성이 존재한다.

반면 Podman은 데몬 없이(demonless) 컨테이너를 직접 실행하는 구조로, 각 컨테이너가 독립적인 프로세스로 동작한다. 특히 rootless 모드를 기본 지원하여 보안성이 뛰어나며, systemd와 자연스럽게 통합되어 운영 환경에 적합하다.

Podman은 Docker CLI와 호환되기 때문에 기존 Docker 명령어를 거의 그대로 사용할 수 있으며, Kubernetes YAML 생성(podman generate kube)과 같은 기능을 통해 컨테이너에서 쿠버네티스로의 확장도 용이하다.

- Docker → Daemon 기반/편의성/생태계 강점
- Podman → Daemonless/보안/쿠버네티스 운영 친화적

Podman 상세 내용

Podman에 관심이 있는 경우, 아래 링크에서 PDF 참고가 가능하다.

https://github.com/tangt64/training_memos/tree/prepare-release-2026/opensource-100